

SKALA

AI Campus 생성형AI

8. AI Agent Capstone

Backend * Vector DB * AI Agent with MCP * Frontend

2026.10

배기주 / SK AX

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX



1. MCP 설계 및 개발
2. 통합 아키텍처
3. Streaming
4. 통합 환경의 운영
5. Agent 확장

AI Campus 생성형AI

1. MCP 설계 및 개발

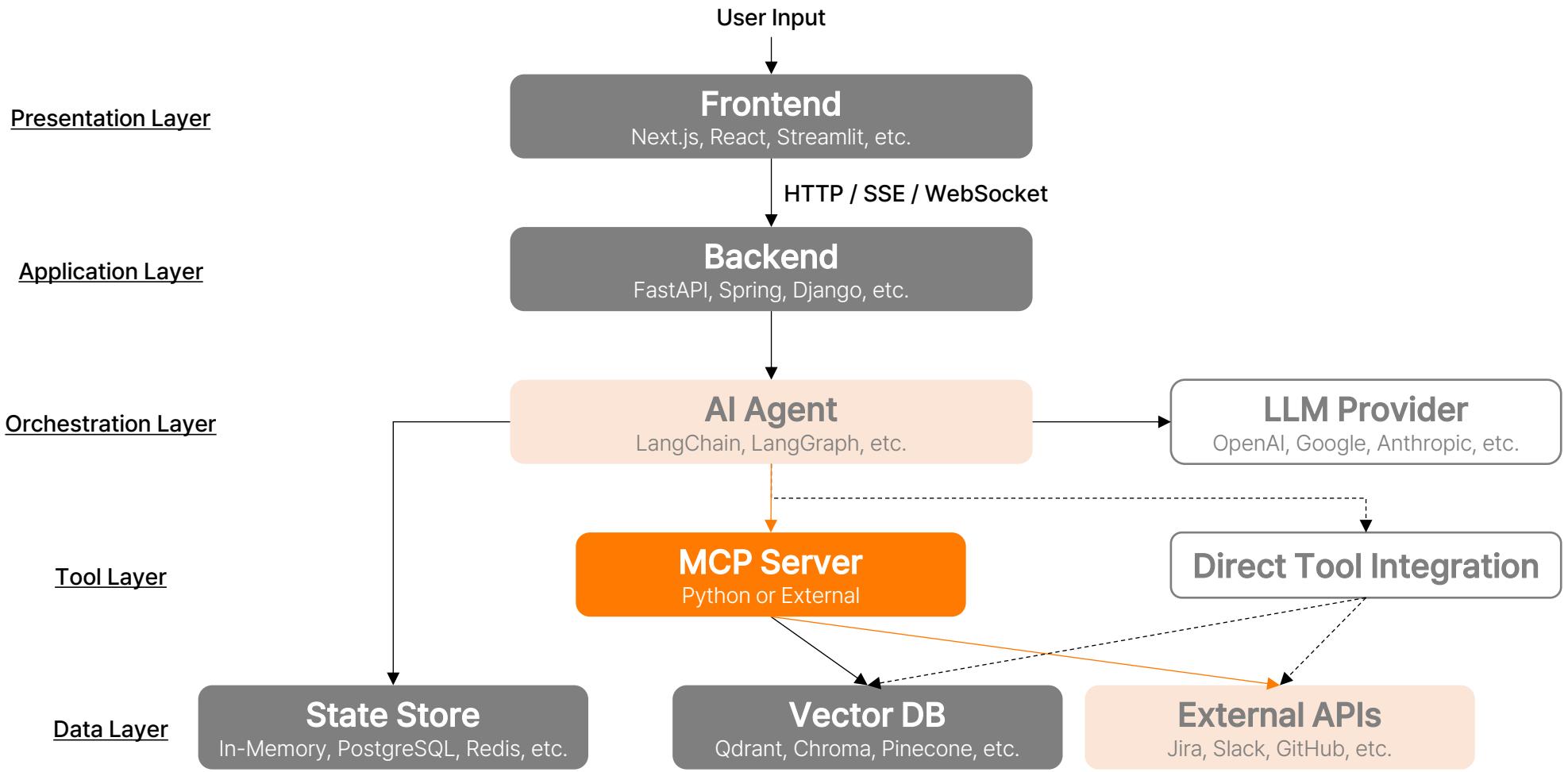
AI Agent MCP

- MCP 설계 : Tool, Resource, Prompt 분리 의사결정
- MCP Server 구현
- MCP Inspector

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

System Diagram



Callback

DONE

- MCP 정의와 등장 배경
- MCP 구성 : Host, Client, Server
- 외부 MCP Server 사용 경험 (e.g., Claude Desktop) ← Optional

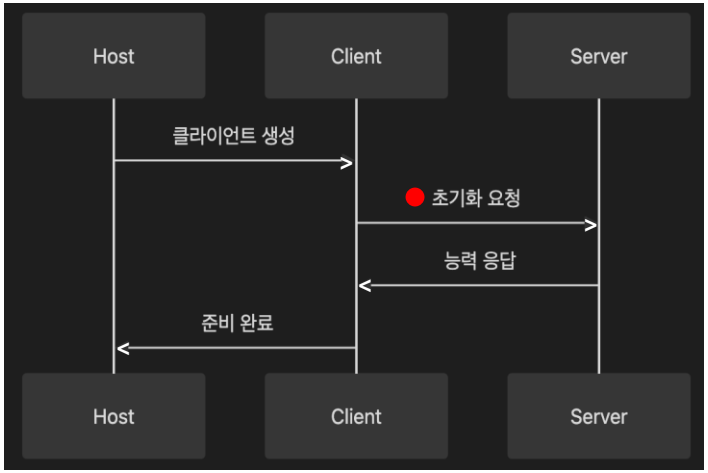
YET

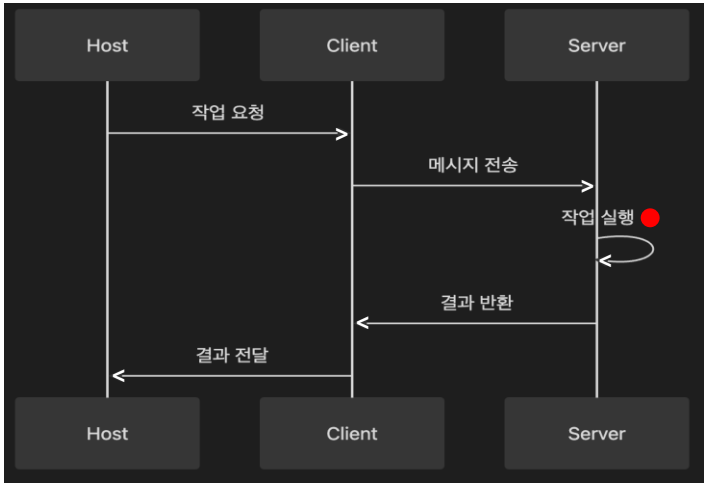
- MCP Server를 처음부터 직접 설계하기
- 어떤 단위로 책임을 쪼개서 노출할 것인가 결정하기
- 도메인에 맞춰 MCP Server 개발하기
- MCP Inspector로 디버깅하기

MCP Architecture

Recap

Host-Client-Server : 명확한 역할과 책임을 가지며, 이를 통해 유연하고 확장 가능한 시스템 구현 가능





MCP Host

- 회사의 팀장처럼, 여러 부서(서버)와 연결을 조율하는 중앙 통제실 역할
- 사용자와 직접 상호 작용하는 어플리케이션
- 주요 역할 :
  - LLM과의 통합 및 전체 데이터 흐름 조율
  - 다양한 서버의 응답을 취합 및 정제하여 AI모델에 전달

MCP Client

- 팀장(Host)과 각 부서(Server) 사이에서 업무 요청/회신을 담당하는 전담 인력
- 호스트 내부에서 동작
- 각 MCP Server와 1:1로 연결되는 중개자 역할을 하는 컴포넌트
- 주요 역할 :
  - 서버와의 독립적인 연결 및 세션 관리
  - 프로토콜 협상, 기능 교환, 요청/응답 메시지 라우팅

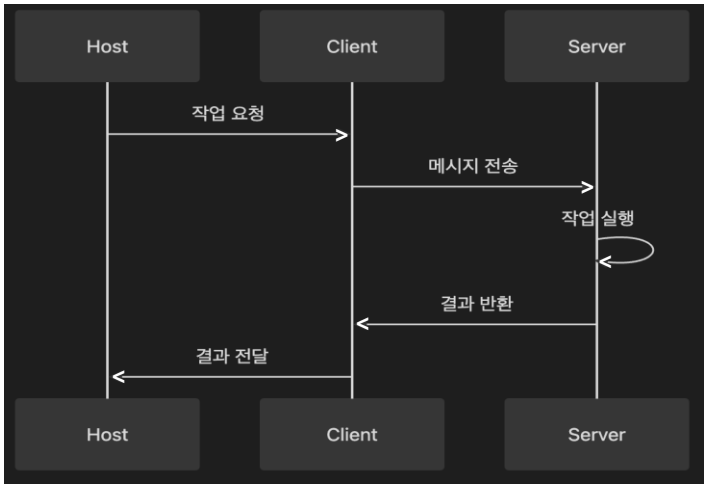
MCP Server

- 회사의 각 부서처럼, 특정 업무(데이터 제공, 도구 실행 등)를 담당
- AI 모델이 사용할 수 있는 3대 핵심 요소(Resources, Tools, Prompts) 제공
- 주요 역할 :
  - MCP 프로토콜 통해 클라이언트의 요청을 받아 정보 제공 또는 작업 실행
  - 로컬 데이터(파일, DB)나 원격 서비스(API, 클라우드 등)와 연결

※ Remark
- Resources : File, API, DB 등 모델이 읽을 수 있는 데이터 컨텍스트
- Tools : 모델이 외부 시스템에 개입하여 실행할 수 있는 액션 (함수 호출)
- Prompts : 사용자가 쉽게 작업을 시작할 수 있도록 정의된 프롬프트 템플릿

Source : https://wikidocs.net/268794, https://modelcontextprotocol.io/specification/2025-06-18

1. MCP 구성 요소



client.py 프로세스

MCP Host
LLM이 호출할 Tool 결정

Connect → Tools → (a specific tool) → Run Tool

MCP Client
Server와 1:1 연결, 호출 전달

```
# Client - MCP Inspector 경우
npx @modelcontextprotocol/inspector@v1-latest \
    uv run python mcp-server/server.py
```

Request / Response

MCP 메시지 (JSON-RPC 2.0) 요청/응답
(Client가 server.py를 하위 프로세스로 실행)

server.py 프로세스

MCP Server

FastMCP 인스턴스,
3대 핵심요소 제공

```
mcp = FastMCP(name="project_name")

@mcp.tool()
@mcp.resource()
@mcp.prompt()

mcp.run()
```

※ 인스턴스

- 설계도 기반으로 실제로 만들어 낸 실물 하나 (e.g. 붕어빵 틀로 찍어 낸 붕어빵 한 개)
- `mcp = FastMCP(...)` : MCP 서버 실물 하나로 만들어 mcp라는 이름을 붙이는 코드
- 이후 mcp에 @mcp.tool() 등으로 기능 등록

2. MCP Server 구성 요소

안전과 일관성 확보 위해 결정 주체를 나누어 정의함

구분	Resource	Tool	Prompt
역할	File, API, DB 등 모델이 읽을 수 있는 Data Context	모델이 외부 시스템에 개입하여 실행할 수 있는 액션 (함수 호출)	사용자가 쉽게 작업을 시작할 수 있도록 정의된 프롬프트 템플릿
Actor	Application(client)-controlled	Model-controlled	User-controlled
Target	Data	Action	Template
Type	• Read-Only • URI로 식별되며 정적이거나 천천히 변하는 Data Context • Client가 자유롭게 list/read 가능	• Read or Write (feat. tool annotation) • 에이전트가 능동적으로 호출하는 함수 • 시점마다 Read 결과가 달라서 정적인 URI로 표현하기 어려움	• N/A • 정적 템플릿
코드	@mcp.resource()	@mcp.tool()	@mcp.prompt()
예시	(e.g.) runbook://incident-response/db-down	(e.g.) create_jira_ticket	(e.g.) incident_analysis_prompt

핵심 차이는 Agent가 결정해서 호출하는가, 미리 노출되어 있는가?

조회성이면 Resource, 능동적 실행이 필요하면 Tool, 사용자가 호출 패턴 위해 필요하면 Prompt

Anti-pattern : 일단 Tool로 만들어 두자

현상 :

- Docstring이 모호하거나 책임 경계가 겹칠 때
- 의미적으로 유사한 Tool이 여러 개 존재할 때 (e.g. : search_docs, find_documents, query_manuals)
- Tool의 input schema가 명확하지 않을 때

주요 부작용 :

- Agent 사고가 흐려짐
  - Tool이 많아질수록 도구 선택 과정에서 delay 발생
  - 불필요한 일까지 모두 도구이면 에이전트 판단 결과가 역으로 모호해 질 수 있음 (+debugging 더 어려워짐)
- 토큰 비용
  - Tool 명세가 System Prompt에 포함되어 토큰 비용 영향
    - OpenAI API : 일정 토큰 수 이상이면 자동 캐싱
    - Anthropic API : `cache_control`을 명시적으로 지정해야 캐싱 적용
    - LangGraph : 내부적으로 호출하는 모델의 캐싱 기능을 얼마나 잘 활용하도록 구성했는지에 달려 있음
  - Resource나 Prompt로 분리 가능한 것까지 Tool이면 비용 증가
- 재사용 어려워짐
  - Tool은 Agent의 LLM 결정 거쳐 호출됨
  - Resource는 Client가 직접 조회, Tool은 에이전트 호출 통해 결정되도록 구분 필요

3. 의사결정 워크플로우

Step 1
Function

"어떤 기능이 필요한가"

- Product가 필요로 하는 기능 또는 역할을 자연어로 모두 작성
- (e.g.) 운영 매뉴얼 검색, Jira ticket 조회, Jira ticket 생성, 장애 진단 가이드 제공, 모니터링 상태 확인
- 분류를 고민하지 말고, 일단 모두 작성 !!!

Step 2
Capability

"어디에 속하는가" – Resource, Tool, Prompt

- 작성한 기능을 맵핑하는 단계
- MCP Server의 Capability로 구분

Action (생성, 수정)이 필요한가?
Yes → **Tool**
No : 미리 노출/정의되어 사용자가 호출하는 템플릿인가?
Yes → **Prompt**
No : 시점마다 결과가 달라지는가? (동적)
Yes → **Tool (Read-Only)**
No → **Resource**

Step 3
Responsibility

"단일 책임 원칙으로 점검: 쪼갤지, 합칠지"

- 하나의 Tool이 여러 책임을 갖지 않는가? ➔ 단일 책임 원칙
- 두 개의 Tool로 나뉘어 있는데, 거의 항상 함께 호출되는가? ➔ 합치기 검토
- 해당 Server가 담당해야 할 범위인가, 다른 Server로 분리할 범위인가? ➔ 도메인 경계 또는 호출 빈도로 판단

함께 해봅시다 !!!

Step 1
Function

ICT 운영 지원 에이전트 기능 :

1. **사내 운영 매뉴얼에서 절차 검색** : 담당자가 문서를 직접 뒤지지 않고, 필요한 운영 절차를 즉시 검색
2. **특정 incident 유형의 runbook 가져오기** : 장애 유형에 맞는 대응 매뉴얼을 바로 불러와 초기 대응 시간을 줄일 수 있도록 함
3. **Jira ticket 검색** : 기존에 등록된 이슈나 작업 현황 조회
4. **Jira ticket 생성** : 새로운 이슈를 티켓으로 등록
5. **모니터링 시스템 상태 조회** : 현재 시스템이 정상 작동 중인지 실시간으로 확인
6. **"장애 진단" 표준 절차 제공** : 장애 발생 시 따라야 할 표준화된 진단 절차 안내

Step 2
Capability

책임	Capability	근거
1. 사내 운영 매뉴얼에서 절차 검색		
2. 특정 incident 유형의 runbook 가져오기		
3. Jira ticket 검색		
4. Jira ticket 생성		
5. 모니터링 시스템 상태 조회		
6. "장애 진단" 표준 절차 제공		

Hands-On

Notion > MCP Server

- ❑ Notion 가이드를 확인하고,
- ❑ MCP Inspector로 함께 결과를 확인 합니다.

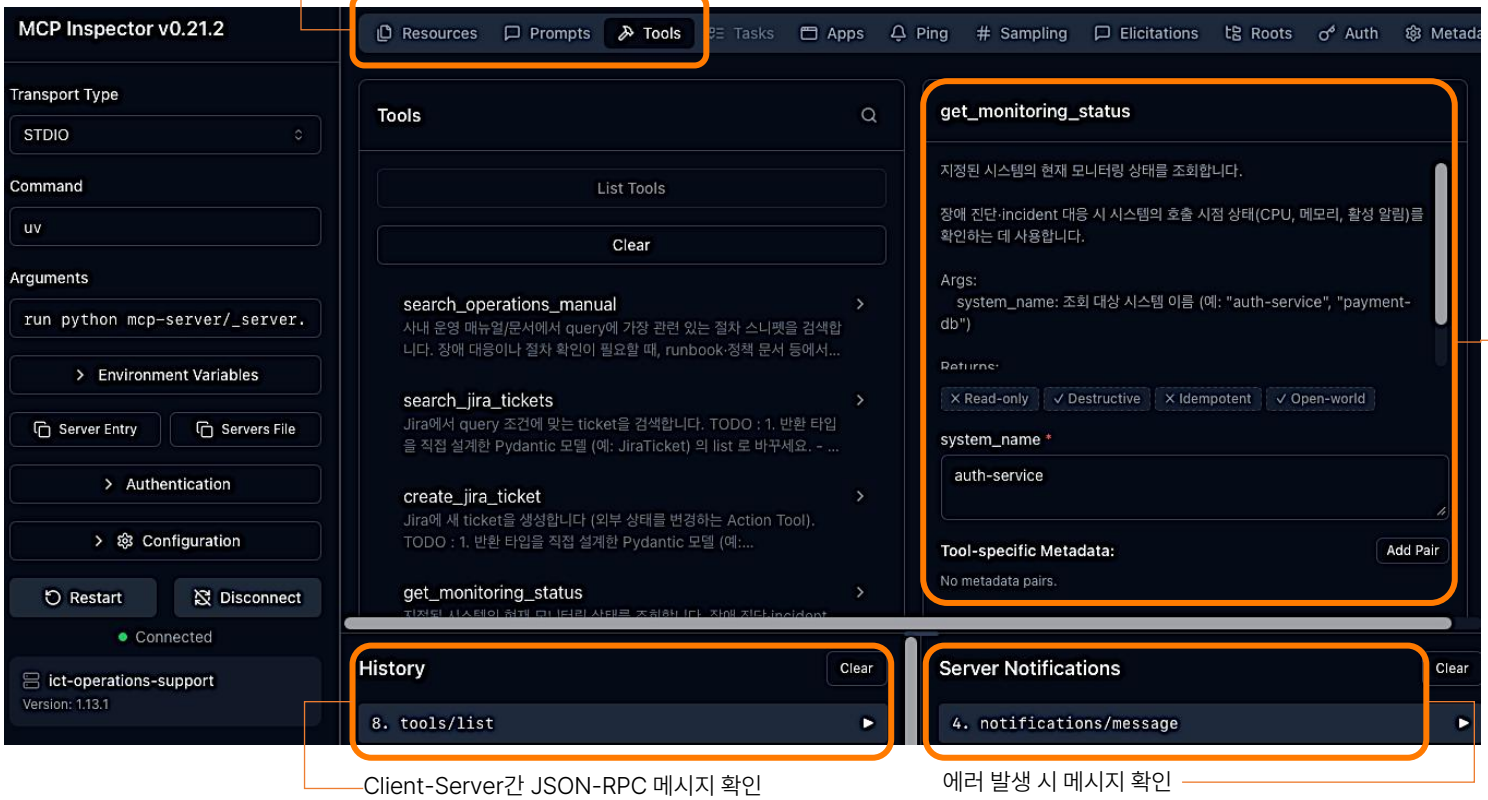
- Notion의 내용을 확인하고 해당 내용을 코드 파일에 반영합니다.
- Inspector에서 정상적으로 출력되는지 확인 합니다.

4. MCP Inspector

MCP Server가 정상적으로 동작하는지 에이전트 연결 이전 단계에 검증하는 도구

- AI가 Tool을 못 쓰는 문제 보다, Tool을 AI가 쓰기 어렵게 만든 문제가 많음
- MCP Server가 정상적으로 동작하는지 사람이 직접 눈으로 확인하고 테스트 할 수 있도록 도와주는 디버깅/검증 도구

Server가 노출하는 Resource, Prompt, Tool 목록 조회



직접 호출 (검증)

- Tool : 인자와 함께 직접 실행
- Resource : URI로 직접 조회
- Prompt : 인자와 함께 렌더링

Client-Server간 JSON-RPC 메시지 확인

에러 발생 시 메시지 확인

4. MCP Inspector

Inspector 통한 자기 점검 체크리스트

Tool 점검

- ❑ 모든 Tool이 Inspector에 보이는가? 누락된 것은 없는가?
- ❑ 각 Tool의 description이 "해당 도구는 언제 쓰는가"를 명확히 표현하고 있는가?
  : 단순히 "X를 한다"가 아니라 "Z 상황에서 X를 한다"
- ❑ Input schema의 각 필드에 description이 있는가?
  : Field(description=...) 누락된 곳은 없는가?
- ❑ Tool을 직접 실행했을 때 의도한 결과가 확인되는가?

Resource 점검

- ❑ URI 패턴이 의도한 대로 동작하는가?
- ❑ `runbook://incident-response/db-down` 같은 구체적인 URI로 조회 가능한가?
- ❑ 존재하지 않는 URI 조회 시 의도한 에러 메시지가 출력되는가?

Prompt 점검

- ❑ Prompt 인자가 명확한가?
- ❑ 인자 누락 시 의도한 에러가 확인되는가?
- ❑ 랜더링된 Prompt에 의도한 워크플로우가 모두 반영되어 있는가?

1. MCP 설계 및 개발

SUMMARY

SUMMARY

2. MCP Server 구성 요소

안전과 일관성 확보 위해 결정 주체를 나누어 정의함

구분	Resource	Tool	Prompt
역할	File, API, DB 등 모델이 읽을 수 있는 Data Context	모델이 외부 시스템에 개입하여 실행할 수 있는 액션 (함수 호출)	사용자가 쉽게 작업을 시작할 수 있도록 정의된 프롬프트 템플릿
Actor	Application(client)-controlled	Model-controlled	User-controlled
Target	Data	Action	Template
Type	• Read-Only • URI로 식별되며 정적이거나 천천히 변하는 Data Context • Client가 자유롭게 list/read 가능	• Read or Write (feat. tool annotation) • 에이전트가 능동적으로 호출하는 함수 • 시점마다 Read 결과가 달라서 정적인 URI로 표현하기 어려움	• N/A • 정적 템플릿
코드	@mcp.resource()	@mcp.tool()	@mcp.prompt()
예시	(e.g.) runbook://incident-response/db-down	(e.g.) create_jira_ticket	(e.g.) incident_analysis_prompt

핵심 차이는 Agent가 결정해서 호출하는가, 미리 노출되어 있는가?

조회성이면 Resource, 능동적 실행이 필요하면 Tool, 사용자가 호출 패턴 위해 필요하면 Prompt

3. 의사결정 워크플로우

SUMMARY

Step 1
Function

"어떤 기능이 필요한가"

- Product가 필요로 하는 기능 또는 역할을 자연어로 모두 작성
- (e.g.) 운영 매뉴얼 검색, Jira ticket 조회, Jira ticket 생성, 장애 진단 가이드 제공, 모니터링 상태 확인
- 분류를 고민하지 말고, 일단 모두 작성 !!!

Step 2
Capability

"어디에 속하는가" – Resource, Tool, Prompt

- 작성한 기능을 맵핑하는 단계
- MCP Server의 Capability로 구분

Action (생성, 수정)이 필요한가?			
	Yes → **Tool**		
	No : 미리 노출/정의되어 사용자가 호출하는 템플릿인가?		
		Yes → **Prompt**	
		No : 시점마다 결과가 달라지는가? (동적)	
			Yes → **Tool (Read-Only)**
			No → **Resource**

Step 3
Responsibility

"단일 책임 원칙으로 점검: 쪼갤지, 합칠지"

- 하나의 Tool이 여러 책임을 갖지 않는가? ➔ 단일 책임 원칙
- 두 개의 Tool로 나뉘어 있는데, 거의 항상 함께 호출되는가? ➔ 합치기 검토
- 해당 Server가 담당해야 할 범위인가, 다른 Server로 분리할 범위인가? ➔ 도메인 경계 또는 호출 빈도로 판단

SUMMARY

4. MCP Inspector

Inspector 통한 자기 점검 체크리스트

Tool 점검

- ❑ 모든 Tool이 Inspector에 보이는가? 누락된 것은 없는가?
- ❑ 각 Tool의 description이 "해당 도구는 언제 쓰는가"를 명확히 표현하고 있는가?
  : 단순히 "X를 한다"가 아니라 "Z 상황에서 X를 한다"
- ❑ Input schema의 각 필드에 description이 있는가?
  : Field(description=...) 누락된 곳은 없는가?
- ❑ Tool을 직접 실행했을 때 의도한 결과가 확인되는가?

Resource 점검

- ❑ URI 패턴이 의도한 대로 동작하는가?
- ❑ `runbook://incident-response/db-down` 같은 구체적인 URI로 조회 가능한가?
- ❑ 존재하지 않는 URI 조회 시 의도한 에러 메시지가 출력되는가?

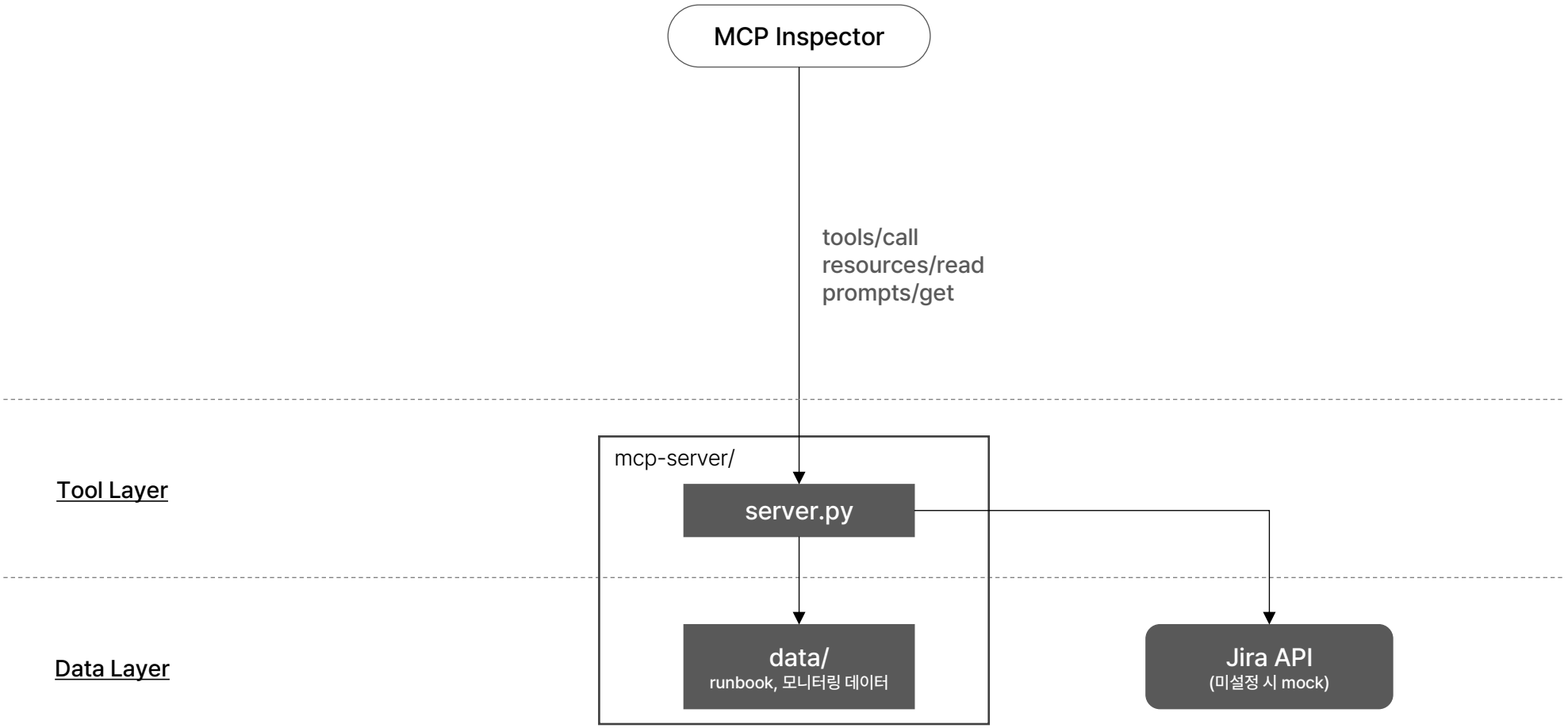
Prompt 점검

- ❑ Prompt 인자가 명확한가?
- ❑ 인자 누락 시 의도한 에러가 확인되는가?
- ❑ 랜더링된 Prompt에 의도한 워크플로우가 모두 반영되어 있는가?

Hands-On

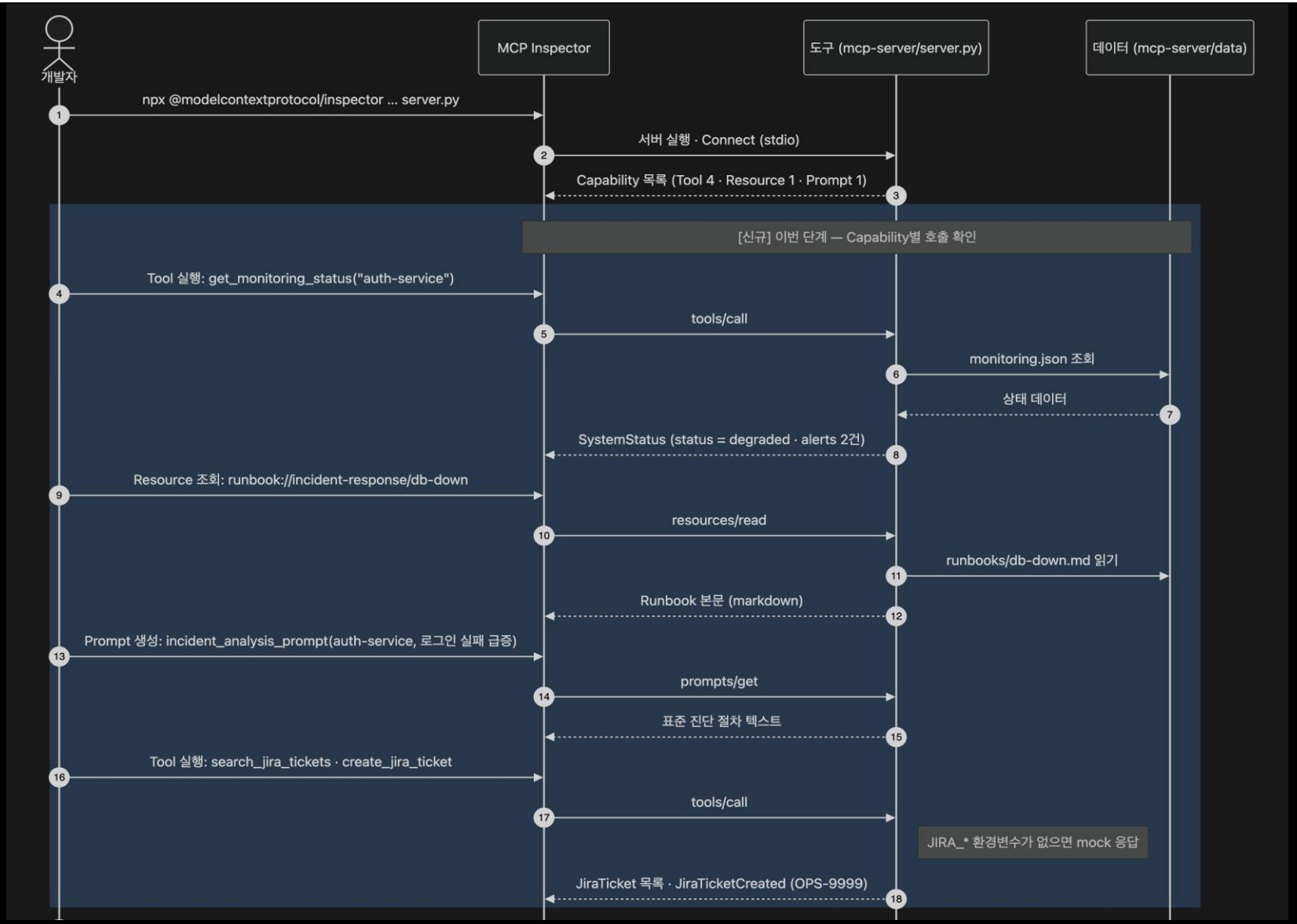
모듈 연결도

MCP Server



서비스 흐름도

MCP Server



AI Campus 생성형AI

2. 통합 아키텍처

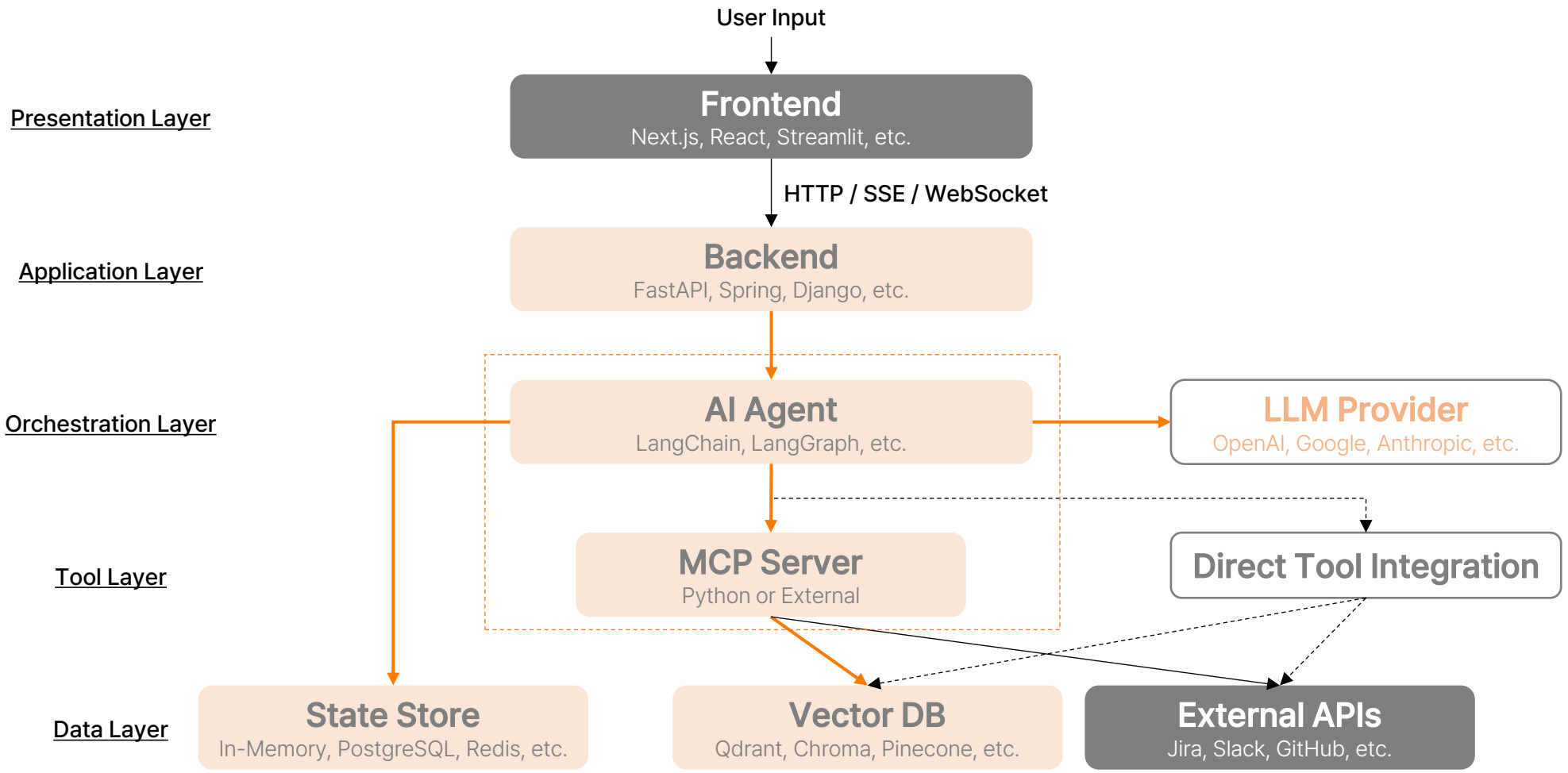


- 시스템 경계 설계
- Fast API에 RAG + Agent + MCP Client 통합
- Thread 관리

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

System Diagram



두 시스템이 동일하게 동작해도

경계가 어떻게 그어졌는가에 따라 완전히 다른 시스템 입니다.

같은 Product이라도 어디서 retrieval 하느냐, 어디서 tool 호출을 결정하느냐, 어디서 session을 관리하느냐에 따라 운영, 디버깅, 확장이 달라집니다.

이 결정은 한 번 결정하면 나중에 바꾸기 정말 어렵습니다.

1. 시스템 경계, System Boundary

용어 정리

개념적 의미

시스템과 그 외부 환경을 구분하는 가상의 선.
시스템 안에 포함되는 것과 포함되지 않는 것을 결정하는 기준.

- 포함/배제 : 무엇이 시스템 내부에 속하고, 무엇이 외부 환경에 속하는가
- 상호작용 지점 : 경계를 통과하는 입력/출력 정의
- 책임 범위 : 시스템이 책임지는 것과 책임지지 않는 것의 구분

SW Engineering

Component / Module / Service가 어디까지 책임지고,
다른 Component와 어떤 방식으로 상호작용 하는가 정의.

- Responsibility Boundary : 해당 Component가 처리해야 할 일과 하지 않아야 하는 일의 구분 → Single Responsibility Principle
- Interface Boundary : Component간 통신 방법과 Contract 정의 (e.g., API Spec, Message Schema, etc.)
- Data Boundary : Component 별 Data Ownership, Read/Write 권한

통합 아키텍처

통합된 시스템에서 각 Component가 어디에 위치하고, 무엇을 책임지며, 어떻게 통신하는가에 대한 의도적 결정의 집합.

- Topology : Component가 어느 프로세스, 서비스에 위치하는가 정의
- Communication : Component간 동기/비동기, 직접 호출/메시징, 통신 프로토콜 정의
- Intentionality : "기술적으로 가능한가"가 아니라 "의도적으로 어디에 둘 것인가"의 결정 → Trade-off로 결정

Remark :

- Component : 시스템을 구성하는 독립적인 기능 단위 (e.g., 로그인 기능, 결제 기능)
- Module : 관련된 코드를 논리적으로 묶어놓은 단위. 하나의 component 내부를 구성하는 코드 파일/폴더 수준의 단위. (e.g., auth.py, payment.py)
- Service : 그 자체로 하나의 완결된 프로그램처럼 동작 (e.g., 결제 서비스 서버)

2. 핵심 결정 사항

AI Agent 중심의 통합 아키텍처 설계 시, 시스템 경계를 정의할 때 결정해야 할 핵심 사항



❑ 1. Retrieval은 누가 하는가?

Backend가 미리 retrieval해서 agent에 전달하는가?

아니면, agent가 tool로 retrieval을 호출하는가?

❑ 2. MCP Server는 어디에 위치하는가?

Agent와 같은 프로세스인가?

별도 프로세스인가? 별도 서비스인가?

❑ 3. Session/State는 어디에서 관리하는가?

Frontend, Backend, Agent, State store 중 어디가 관리의 주체인가?

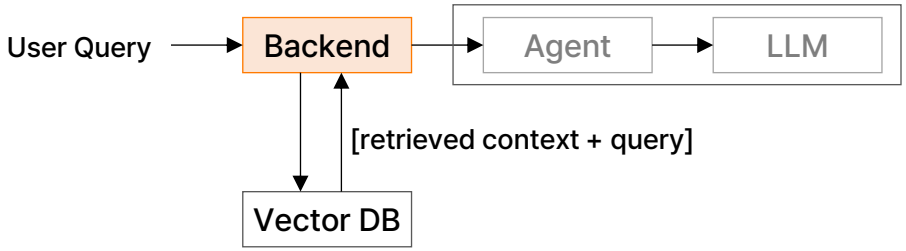
❑ 4. 에러는 어디서 어떻게 처리하는가?

각 Layer에서 어떻게 실패처리 하는가?

그 때 어디서 graceful degradation을 하는가?

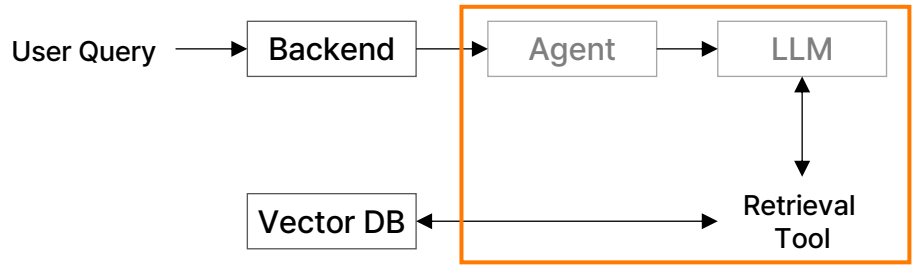
1. Retrieval은 누가 하는가?

(1A) Back



- Backend가 항상 Retrieval 수행
- Agent는 이미 Context를 받은 상태에서 Reasoning만 수행
- Agent는 Retrieval Tool 호출 결정을 내릴 필요 없음

(1B) Agent

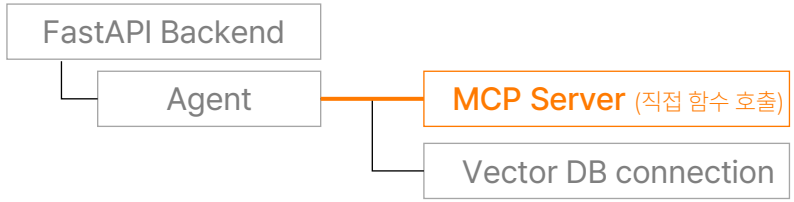


- Agent가 필요할 때 Retrieval Tool 호출
- Agent가 tool calling 결정에 retrieve 포함
- Multi-step Reasoning 과정 시 여러 번 Retrieve 가능

결정	결정 사항 없음. 항상 Retrieve 수행	Agent 판단 하에 Retrieve 수행 결정
유연성	낮음. 필요한 Context를 Backend가 챙겨주어야만 함	높음. 필요 시 Agent가 여러 번 수행할 수 있음.
적합한 상황	• Latency가 매우 중요한 Product • 비용 예측이 중요한 Enterprise 환경 • 검색이 선택이 아닌 필수 도메인 (e.g., 의약품, 법률 등) • 사용자별 검색 가능 범위가 다른 경우	• 하나의 업무 처리 위해 여러 데이터 소스를 상황에 따라 조합해야 하는 경우 : (e.g.) 매뉴얼 검색 + Jira 조회 + 실시간 모니터링 조회

2. MCP Server는 어디에 위치하는가?

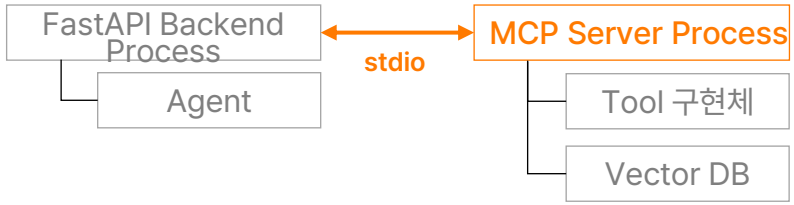
(2A) In-process



Agent와 같은 Python Process (in-process)

- MCP Server는 별도 process로 띄우지 않음
- Tool 함수를 Agent가 직접 import 해서 호출
- 사실상 MCP를 사용하지 않음 (= MCP Protocol 통신 없음)

(2B) Sub-process

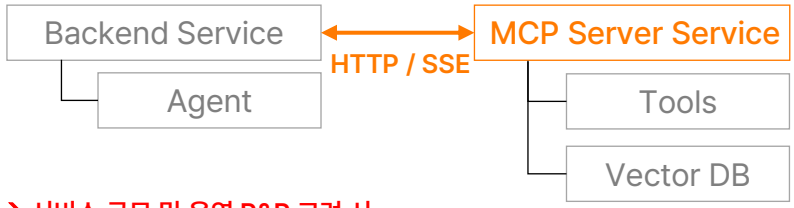


→ MCP 운영을 같은 머신에서 관리할 수 있음

별도 Process, 같은 서버 (subprocess / sidecar)

- MCP Server를 Backend Process가 subprocess로 띄움
- stdio transport
- 같은 머신에서 동작

(2C) Network



→ 서비스 규모 및 운영 R&R 고려 시...

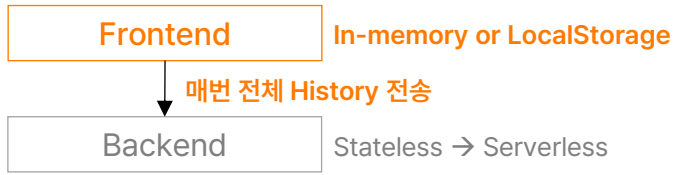
- 여러 Backend Service가 같은 MCP Server를 공유해야 할 때
- MCP Server를 다른 팀이 운영할 때
- Production에서 독립 배포 및 확장 필요

별도 Service (Network)

- MCP Server가 독립 서비스로 배포
- HTTP/SSE transport
- 다른 머신에서 동작 가능, 여러 Backend가 공유 가능

3. Session/State는 어디에서 관리하는가?

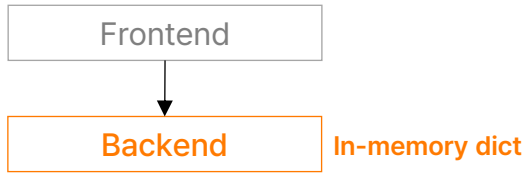
(3A) Frontend



Frontend에서 관리

- 모든 대화 이력을 Fronted가 들고 있음
- Backend는 매 요청마다 Frontend에서 받은 History로 작업

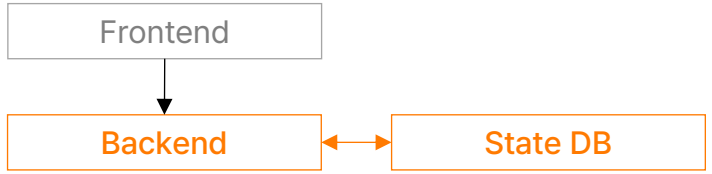
(3B) Backend & In-memory



Backend에서 관리, In-memory

- Backend가 thread_id 기반으로 메모리에 보관
- 서버 재시작 시 손실

(3C) Backend & DB



Backend에서 관리, State DB (persistent)

- Backend가 thread_id 기반으로 Store에 저장
- LangGraph checkpointer로 구현

4. 에러는 어디서 어떻게 처리되는가?

Layer	예상 가능한 에러 케이스
Frontend	• 네트워크 끊김 • Browser Refresh • SSE 연결 끊김
Backend	• Service Unavailable • Timeout • 동시성 문제
Agent	• LLM API 에러 • Rate Limit • Response 형식 오류 • Graph State 오류 • Conditional Edge 분기 실패
MCP Server	• Subprocess down • Schema validation 오류
Vector DB	• 접속 끊김 • Query timeout • Result not found
State DB	• Connection pool소진 • Transaction 실패

(4A) **가장 가까운 Layer에서 처리, 위로 Graceful Messaging**

Vector DB 실패 → MCP Server catch → "no results found" 응답 → Agent 다음 단계 실행

- 장점 : 사용자 경험 매끄러워 짐
- 단점 : 진짜 문제가 숨겨질 수 있음

(4B) **위로 전파, 최상위 Layer에서 일괄 처리**

Vector DB 실패 → 그대로 전파 → Backend catch → 사용자 에러 메시지 출력

- 장점 : 디버깅 쉬움, 명확한 문제 파악 용이
- 단점 : 사용자 경험 어색

(4C) **Layer별 Retry / Fallback**

Vector DB 실패 → MCP Server, Retry → Failure catch & Mock data fallback → 사용자 전달

- 장점 : 서비스 안정성 확보
- 단점 : 프로세스 복잡, fallback 처리 되어 사용자가 원하는 답변이 아닐 수 있음

Product 설계 시 결정해야 할 사항

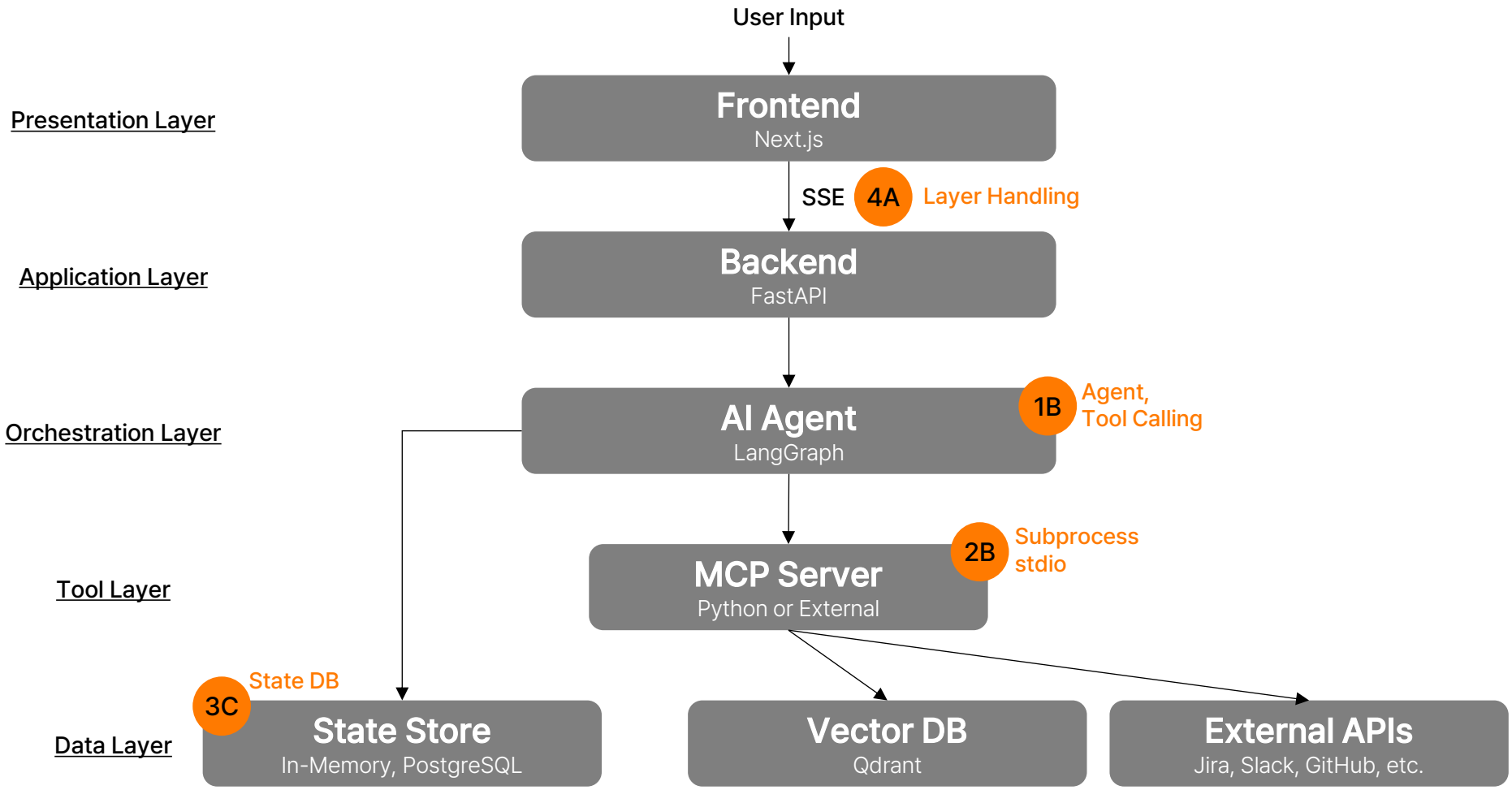
- ❑ Q1. Retrieval(외부 정보 조회)는 Backend가 미리 할 것인가, Agent가 Tool로 호출할 것인가?
- ❑ Q2. 외부시스템(MCP, API)은 같은 프로세스인가, 별도 서비스인가?
- ❑ Q3. Session/State는 어디에 두고, 어떻게 관리할 것인가?
- ❑ Q4. 각 Layer가 어떻게 실패하고, 어디서 graceful degradation 할 것인가?

위 사항에는 정답이 없습니다.

Product에 따라 근거 기반으로 결정하면 됩니다.

System Diagram

for Capstone



4. Thread 관리

LangGraph에서 대화의 영속성을 만드는 두 가지 추상화

Thread :

하나의 대화 단위를 식별하는 논리적 개념.
LangGraph에서 state를 격리/관리하는 최소 단위

```
config = {
    "configurable": {"thread_id" : "abc123"}
}
```

- 식별자 : thread_id (일반적으로 UUID v4)
- 격리 단위 : thread_id가 다르면 서로 섞이지 않음
- Owner : Client(Front) 또는 Server 중 어디서 thread_id 관리 주체가 될 것인지 결정 필요

Checkpointer :

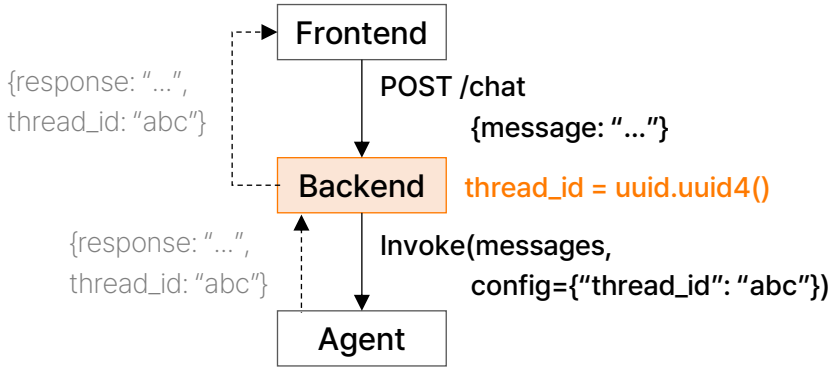
Thread state를 저장/조회하는 추상화 인터페이스.
LangGraph state의 영속성 담당

- 단위 : Graph의 매 step 마다 checkpoint 생성
- Checkpoint 내용 : 특정 시점의 thread state snapshot
  - 대화 메시지 : HumanMessage, AIMessage, ToolMessage
  - Tool Call 이력
- State 저장 : agent의 매 step 이후 자동으로 store에 저장 (MemorySaver)

5. Thread Owner 결정

Client 또는 Server 중 어디서 thread_id 관리 주체가 될 것인지 결정 필요

(A) Backend



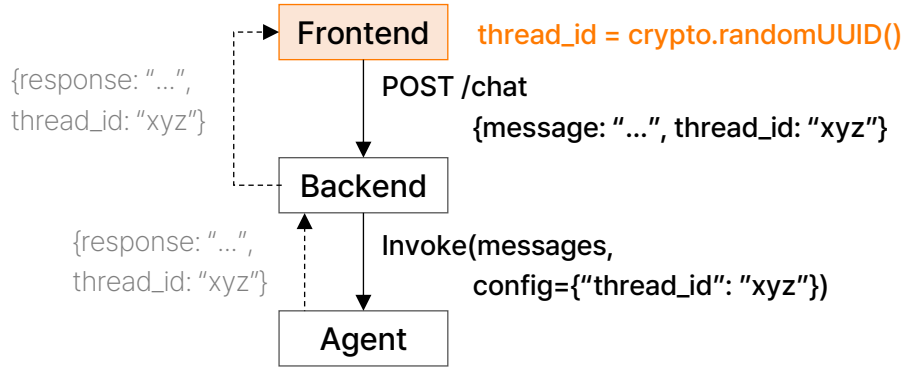
특징 :

- Frontend는 thread_id 모름
- Backend가 매번 새로운 요청이면 ID 생성
- Backend가 응답에 thread_id 포함해서 반환
- Frontend는 전달받은 thread_id를 다음 요청에 다시 보냄

Backend가 해야 할 추가적인 일 :

- thread_id 판별 로직 : 기존 id? 새로운 id?
- 사용자별 active thread 추적 필요 (또는 frontend가 매번 thread_id 전달)

(B) Frontend



특징 :

- Frontend는 새로운 대화 시작 시 UUID 생성 (e.g. "새로운 대화" 버튼)
- Frontend가 자신의 thread 목록을 직접 관리
- Backend는 전달받은 thread_id를 그대로 사용
- Backend는 stateless에 가까움

Frontend가 해야 할 추가적인 일 :

- 사용자별 thread 목록 관리 (localStorage 또는 backend 조회)
- UUID 생성 로직

5. Thread Owner 결정

구분	(A) Backend	(B) Frontend	고려사항
Backend State 관리	• Backend가 active thread를 추적해야 함 → Stateful	• Backend는 받은 thread_id를 그대로 사용 → Stateless	• Backend 복잡도 (+ Scalability)
Frontend Refresh	• Front Refresh 되면 thread_id 잃어버림 • Backend에 “마지막 thread” 요청 필요 (또는 새로운 thread로 시작)	• Front가 localStorage에 보관하면 Refresh 후에도 동일한 thread로 이어가기 가능	
Thread Lifecycle	• Backend 로직이 결정 • New vs Existing session 판별 필요	• Frontend가 UI 이벤트에 따라 직접 결정 • (e.g.) “새로운 대화” 버튼	• UI/UX 설계
Client 다양성	• Frontend 외 다양한 Client(CLI, script, mobile app)에서도 자연스럽게 동작 • 다른 Client 또는 다양한 Device에서 조회 가능	• 모든 Client가 UUID 생성 로직을 가져야 함 • Multi-Device 동기화가 필요한 경우 별도 처리 필요 (e.g. 메타데이터 관리)	• API 일반성 • 다양한 Client 지원 여부 • Multi-Device & Multi-turn 처리 지원
인증 처리	• Backend가 인증된 사용자의 thread를 자연스럽게 추적	• Frontend가 보낸 thread_id 소유권을 backend가 매번 검증 필요	• Multi-user 환경의 Security 측면
분산 시스템	• Backend가 active thread를 추적하면 instance 간 thread 상태 공유 필요	• Backend가 Stateless이므로 load balance 뒤에 여러 instance 자유롭게 배치 가능	• Production-grade • 배포 적합도

[참고] RFC 4122 UUID v4

thread_id = uuid.uuid4()
thread_id = crypto.randomUUID()

표준을 따르는 같은 형식의 식별자를 생성하는 것으로 실행 환경, 반환 타입, 보안 요구사항에 차이 있음

- UUID v4 : (xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx)형식, 36자
- Cryptographically secure 난수 사용 (예측 불가)
- Frontend가 crypto.randomUUID(...)로 만든 값을 Backend가 받아 uuid.UUID(...)로 파싱 가능. 반대도 동일

구분	uuid.uuid4()	crypto.randomUUID()
실행 환경	Python runtime (Backend)	Browser + Node.js 18+
표준	Python stdlib uuid 모듈	Web Crypto API
난수 소스	os.urandom()	Browser/V8 CSPRNG
반환 타입	UUID 객체 (str() 변환 필요)	string
Secure Context 요구	없음	HTTPS 또는 localhost 필수

※ 반환 타입 처리
- Python의 uuid.uuid4()는 UUID 객체를 반환하므로
- Dict의 key로 쓰거나 JSON 직렬화 시 명시적 변환 필요

```
thread_id = uuid.uuid4()
config = {"thread_id": str(thread_id)}
```

※ IP로 접근하면 에러 발생 – crypto is undefined
해결책 :
- http://localhost:3000 또는 https:// 사용
- 사내망 테스트 : HTTPS 인증서 설정 필요
- Fallback : uuid npm 패키지
  (import { v4 as randomUUID } from 'uuid')

2. 통합 아키텍처

SUMMARY

SUMMARY

2. 핵심 결정 사항

AI Agent 중심의 통합 아키텍처 설계 시, 시스템 경계를 정의할 때 결정해야 할 핵심 사항

❑ 1. Retrieval은 누가 하는가?

Backend가 미리 retrieval해서 agent에 전달하는가?

아니면, agent가 tool로 retrieval을 호출하는가?

❑ 2. MCP Server는 어디에 위치하는가?

Agent와 같은 프로세스인가?

별도 프로세스인가? 별도 서비스인가?

❑ 3. Session/State는 어디에서 관리하는가?

Frontend, Backend, Agent, State store 중 어디가 관리의 주체인가?

❑ 4. 에러는 어디서 어떻게 처리하는가?

각 Layer에서 어떻게 실패처리 하는가?

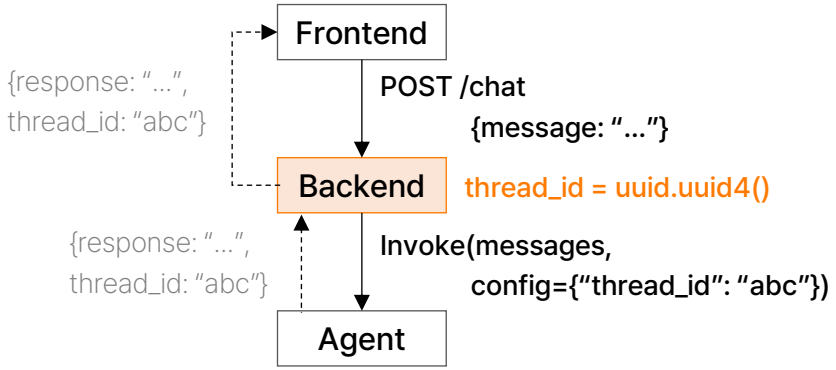
그 때 어디서 graceful degradation을 하는가?

SUMMARY

5. Thread Owner 결정

Client 또는 Server 중 어디서 thread_id 관리 주체가 될 것인지 결정 필요

(A) Backend



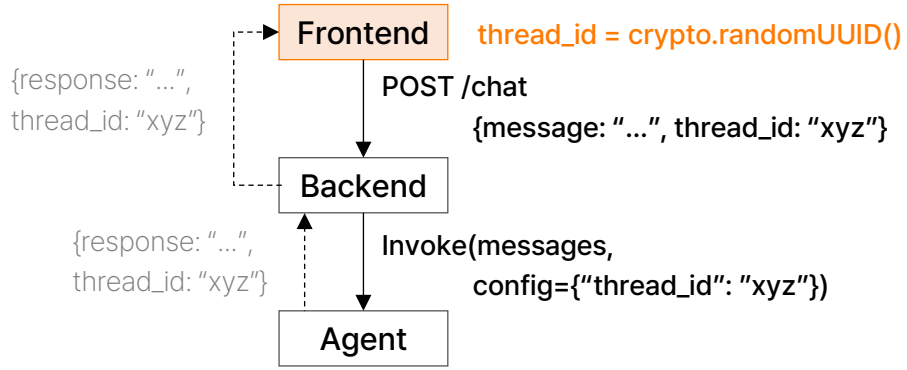
특징 :

- Frontend는 thread_id 모름
- Backend가 매번 새로운 요청이면 ID 생성
- Backend가 응답에 thread_id 포함해서 반환
- Frontend는 전달받은 thread_id를 다음 요청에 다시 보냄

Backend가 해야 할 추가적인 일 :

- thread_id 판별 로직 : 기존 id? 새로운 id?
- 사용자별 active thread 추적 필요 (또는 frontend가 매번 thread_id 전달)

(B) Frontend



특징 :

- Frontend는 새로운 대화 시작 시 UUID 생성 (e.g. "새로운 대화" 버튼)
- Frontend가 자신의 thread 목록을 직접 관리
- Backend는 전달받은 thread_id를 그대로 사용
- Backend는 stateless에 가까움

Frontend가 해야 할 추가적인 일 :

- 사용자별 thread 목록 관리 (localStorage 또는 backend 조회)
- UUID 생성 로직

Hands-On

Notion > FastAPI 통합

- ❑ Notion 가이드를 확인하고,
- ❑ terminal로 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인합니다.

Hands-On

[Remind] FastAPI

FastAPI는 외부 요청을 받아 처리하고, 그 결과를 돌려주는 Backend 창구를 만드는 도구

- 사용자 (또는 화면)가 Agent 기능을 사용하려면, 요청을 받아주는 "창구"가 필요
- FastAPI는 파이썬으로 창구를 빠르고 간단하게 만들 수 있음
- (비유) 손님 → 주문서 → 주문 창구(FastAPI) → 주방 → 창구에서 음식을 손님에게 전달

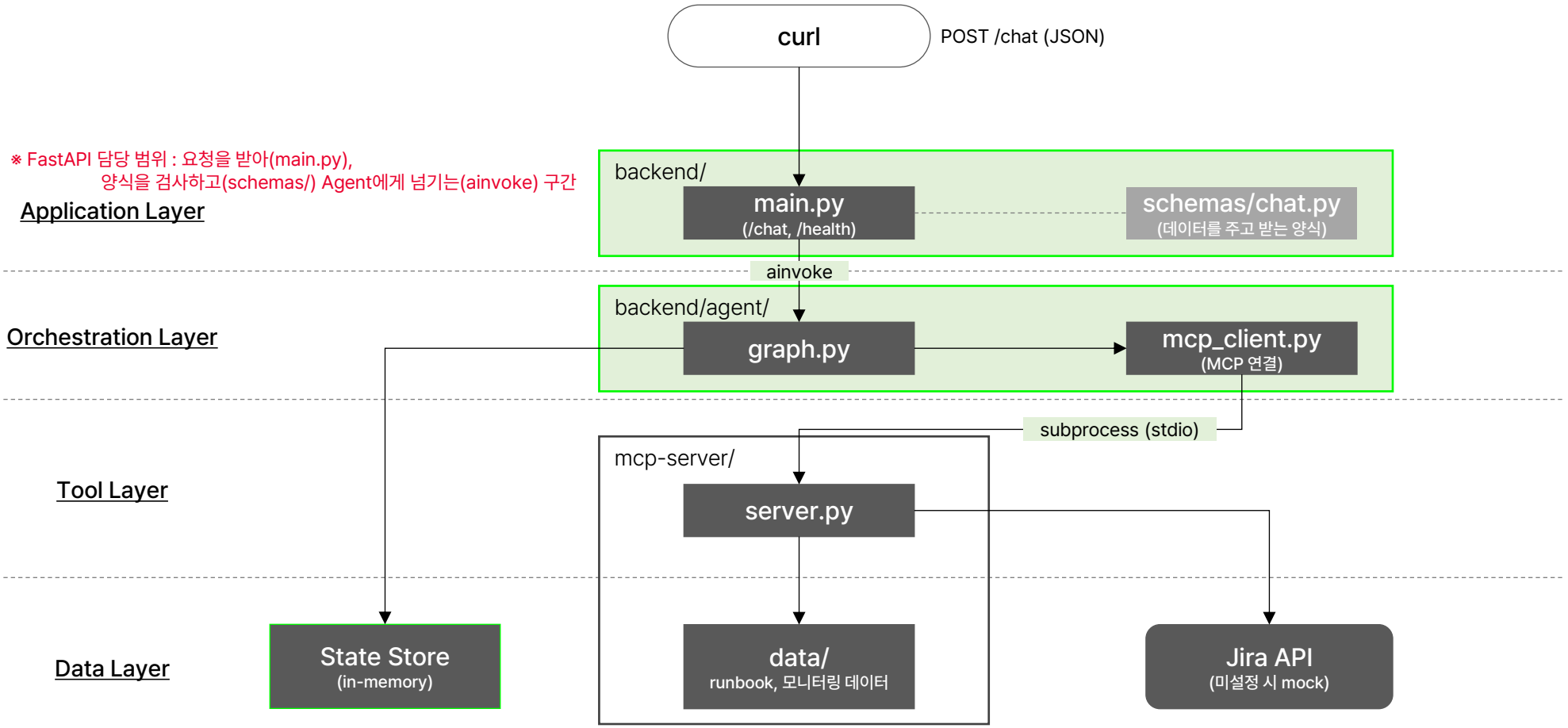
기본 개념 :

- **요청/응답 :** 손님이 주문하고(요청), 창구가 결과를 돌려줌(응답)
- **GET/POST :** 조회만 할 때는 GET, 데이터를 보내 처리를 맡길 때에는 POST
- **JSON :** 주문서를 적는 공통 표기법
- **Schema :** 주문서 양식
- **Endpoint :** 업무별 접수 주소. 업무 마다 주소가 다름 (e.g. 대화는 `/chat`, 상태 확인은 `/health`)

Hands-On

모듈 연결도

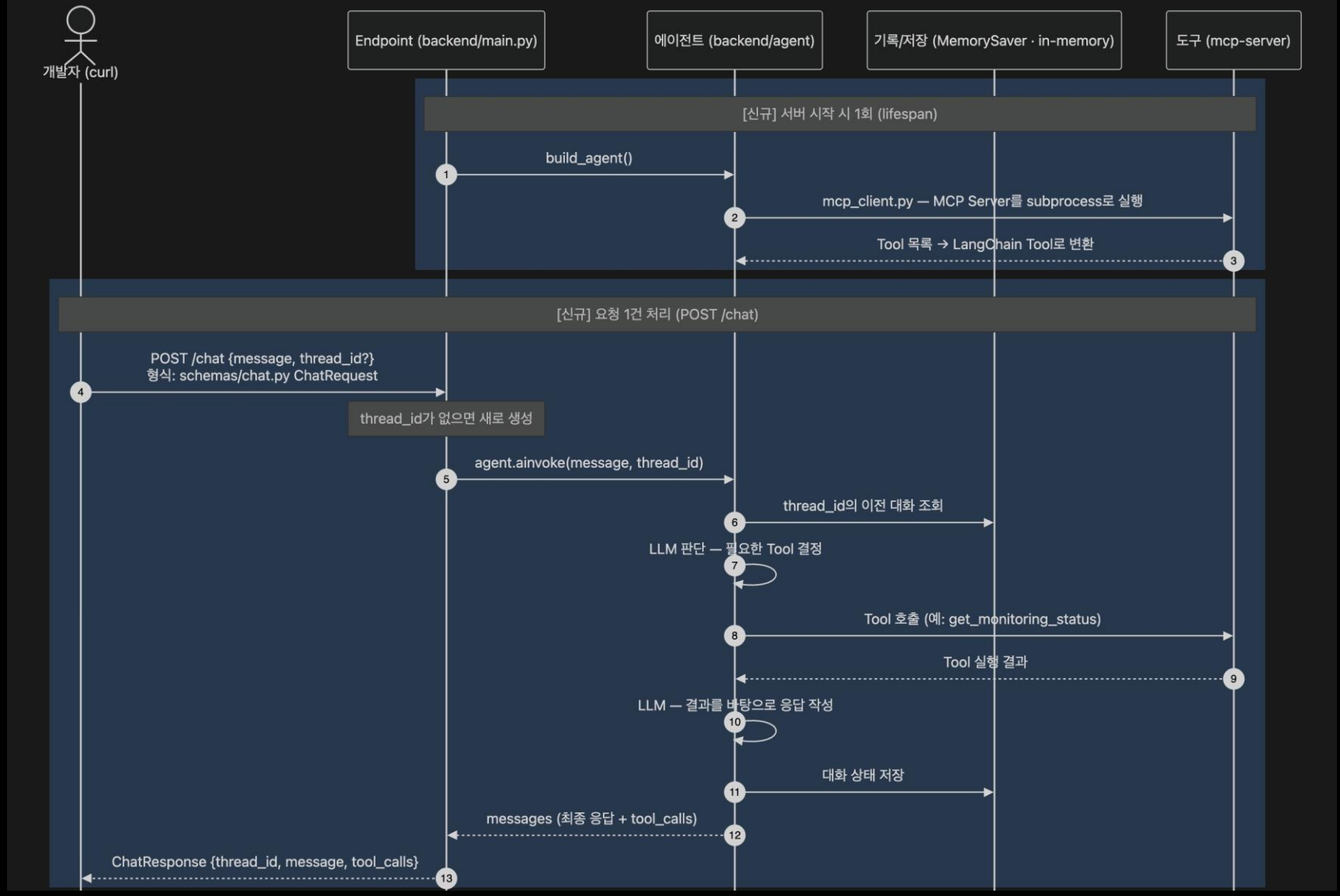
FastAPI Integration



Hands-On

서비스 흐름도

FastAPI Integration



AI Campus 생성형AI

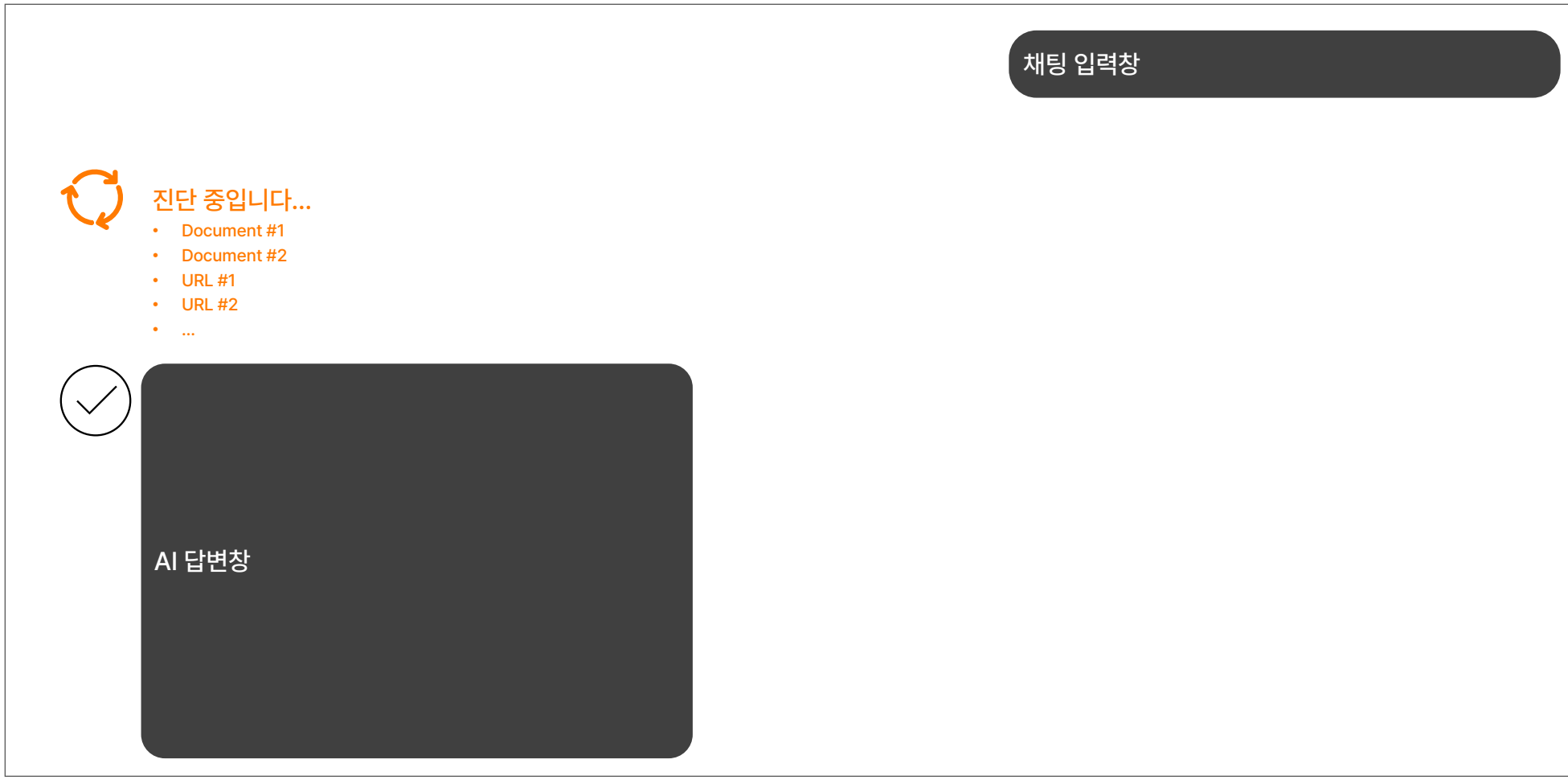
3. Streaming Frontend Backend

- Streaming 위한 3가지 차원 : Transport, Content, Lifecycle
- Backend : SSE Endpoint
- Frontend : Next.js + Vercel AI SDK

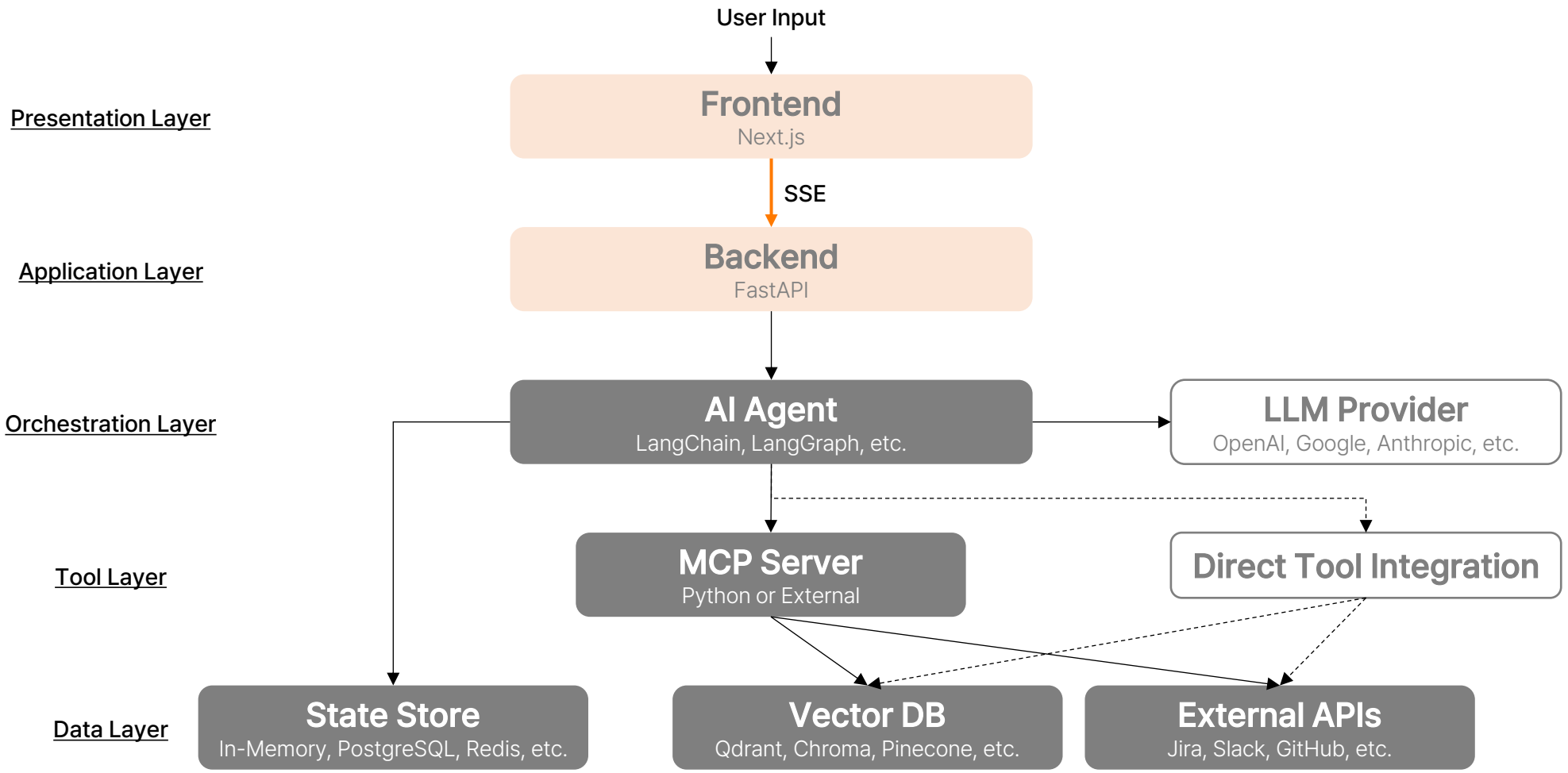
본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

Mockup Image



System Diagram



Callback

DONE

- Agent + Tool Calling
- LangGraph .invoke() : Agent 호출 + 최종 응답 반환
- LangGraph .stream() / .astream() : Backend 내부에서 노드 단위로 yield
- FastAPI + Agent + MCP 통합 : 동기 Endpoint로 최종 응답만 전달

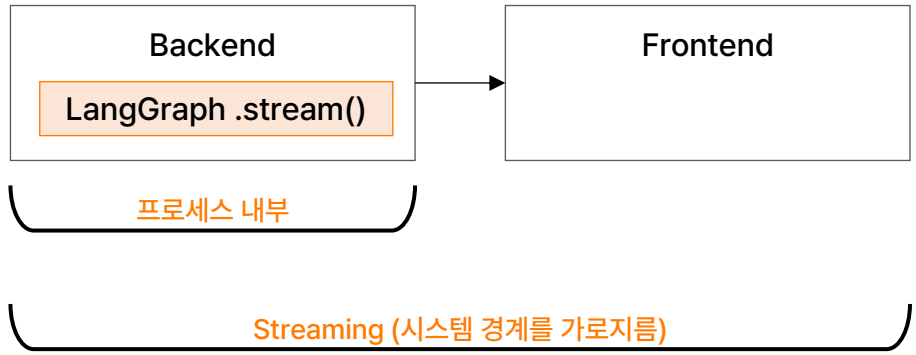
YET

"stream을 HTTP 채널로 흘려보내자"

- Token 단위 + Tool Call 진행 상황을 사용자에게 가시화
- LangGraph event를 SSE Event로 변환
- FastAPI에 SSE Endpoint 추가 + Event Schema 설계
- Frontend Chat UI 구현

Stream vs Streaming

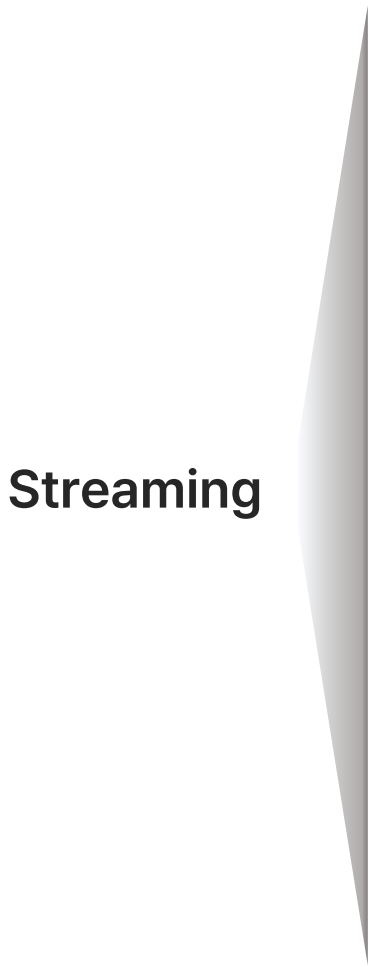
용어 정리



구분	LangGraph .stream() / .astream()	Streaming
개요	LangGraph, Graph 객체의 메서드	시스템 간 점진적 데이터 전달 패턴
경계	단일 프로세스 내부	프로세스 / 머신 / 네트워크를 가로지름
전달 채널	Python async iterator	HTTP 기반 프로토콜 (SSE / WebSocket / chunked)
데이터 단위	Graph node 실행 결과	의미 있는 이벤트 (token, tool call, planning, error 등)
수신자	같은 프로세스 안의 호출자 (Python 코드)	다른 시스템 (브라우저, Client)
Streaming 관점	Streaming Source	Delivery infrastructure

1. Streaming 위한 3가지 차원

3가지 독립적인 결정의 조합



1 Transport

어떻게 streaming 채널을 만드는가?

- HTTP Streaming (chunked transfer)
- Server-Sent Events (SSE)
- WebSocket
- gRPC streaming

2 Content

무엇을 streaming할 것인가?

- Text token : LLM 응답 글자
- Structured event : tool call 시작, 종료, 결과
- State delta : agent state 변화
- Custom payload : Product 특화 이벤트

3 Lifecycle

언제 streaming이 시작되고 끝나는가?

- 요청 시작을 Streaming 시작으로
- 종료 신호 처리
- 에러 시 처리
- Client 끊김 시 처리

1-(1). Transport

Client와 Server가 어떻게 streaming 채널을 만드는가?

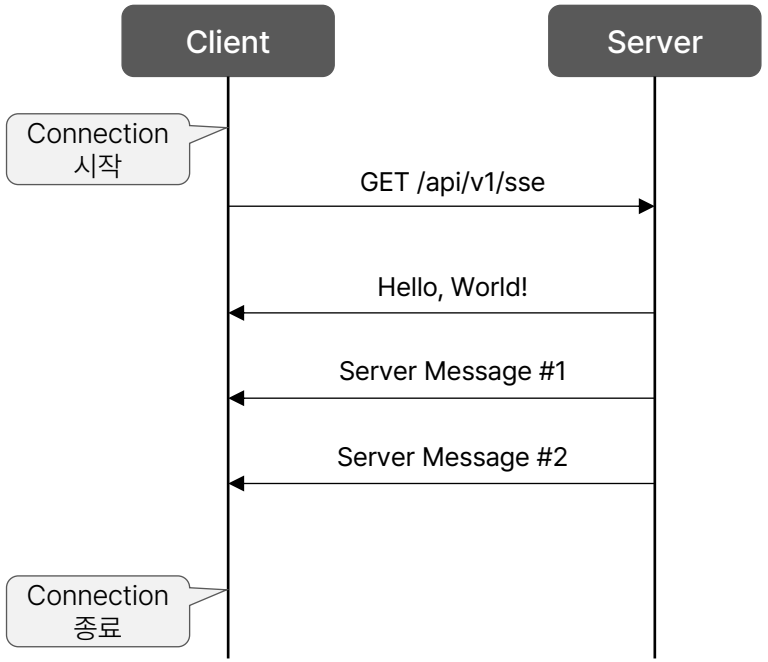
구분	HTTP Streaming	SSE	WebSocket	gRPC streaming
방향	• 단방향 (Server → Client)	• 단방향 (Server → Client)	• 양방향	• 양방향 가능
프로토콜	• HTTP/1.1	• HTTP	• WS protocol	• HTTP/2
특징	• 가장 단순. Response body를 청크로 흘려보냄 • Client는 청크를 받는 대로 처리	• HTTP 위에서 동작 • Event-based 구조	• 완전한 양방향 통신 • Stateful connection	• Protocol Buffers, 타입 안전 • 양방향 streaming 지원
단점	• 이벤트 구조화 약함	• 단방향	• HTTP 의존성이 적어 인프라 (proxy, load balancer)와의 호환성 주의	• Browser Client 사용 제한 (gRPC-Web 필요)
주요 용도	• 단순 응답 streaming	• AI Chat • Notification	• 실시간 협업 • 게임	• Microservices 간 통신

왜 SSE (Server-Sent Events)가 AI Chat의 표준이 되었는가? :

- HTTP 기반 : 기존 인프라와 자연스럽게 호환
- 단방향 충분 : AI Chat은 사용자 요청 후 응답을 받는 패턴. 양방향 필요성 낮음
- Event-based 구조 : 다양한 종류의 이벤트(token, tool_call, error)를 자연스럽게 표현 가능
- 간단한 구현 : Server는 HTTP Response에 특정 형식으로 데이터를 흘려보내면 됨

[참고] SSE 동작 방식

- SSE는 Server에서 Client로의 일방향 통신 채널을 HTTP 통해 설정함
- 일반 HTTP처럼 요청은 한번 보내고, 응답은 연결을 유지한 채 여러 번 나눠 받음



SSE 메시지 형식 (예시) :

```
event: token
data: {"content": "안녕"}
```

Payload

화면에 보여 줄 실제 내용
(메시지에서 꼬리표 뗀 알맹이)

```
event: token
data: {"content": "하세요"}

event: tool_call_start
data: {"tool_name": "search_manual", "args": {...}}

event: done
data: {"thread_id": "..."}
```

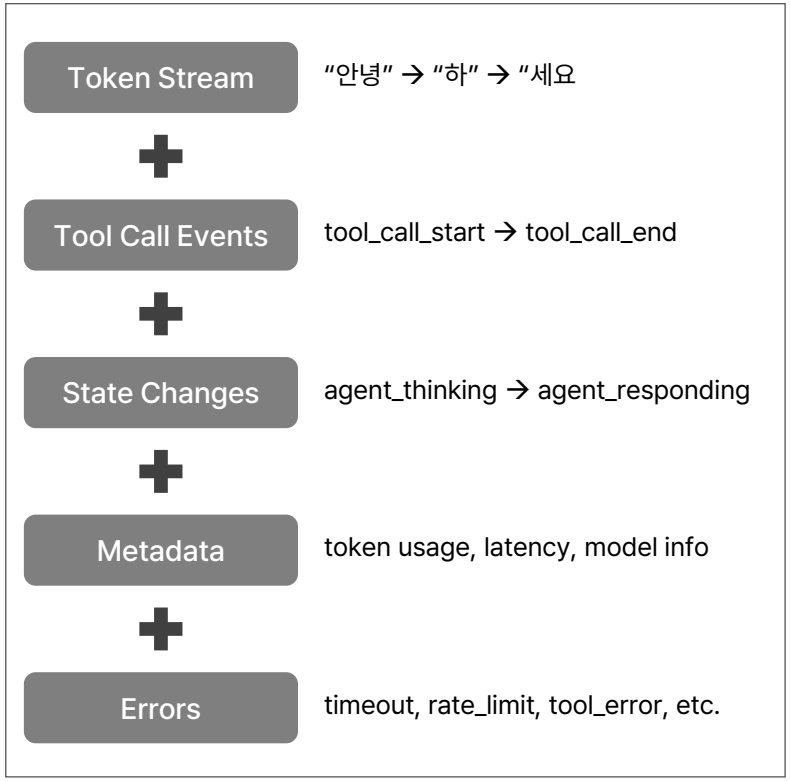
SSE 한계점 :

- 단방향이라 stream 중 Human이 끼어들기(interrupt) 어려움
- HTTP/1.1 connection 제한 (도메인(origin)당 6개). HTTP/2에서 대부분 완화
- 일부 proxy/load balancer에서 buffering 이슈

1-(2). Content

무엇을 streaming 할 것인가?

AI Agent Streaming의 다층 구조 :



- AI Agent streaming은 Multi-Event Streaming
- 단순 텍스트를 스트리밍하면 사용자가 agent의 사고 과정을 확인할 수 없음
- Tool call이 진행 중임을 보여주지 않으면 사용자는 프로세스가 멈추었다고 인식
- UI/UX 통해 agent의 모든 단계를 가시화 하는 것이 필요

[참고] Event Schema 설계

- Type Field 명확 : 모든 이벤트에 type 또는 event 필드로 구분
- Payload가 type별로 구조화 : type에 따라 데이터 형식 명확화
- Type-Safe : TypeScript, Pydantic 등으로 Client/Server 모두에서 타입 검증
- Forward-Compatible : 추후 새로운 이벤트 type 추가해도 기존 Client 코드가 깨지지 않도록 설계

나쁜 예시 :

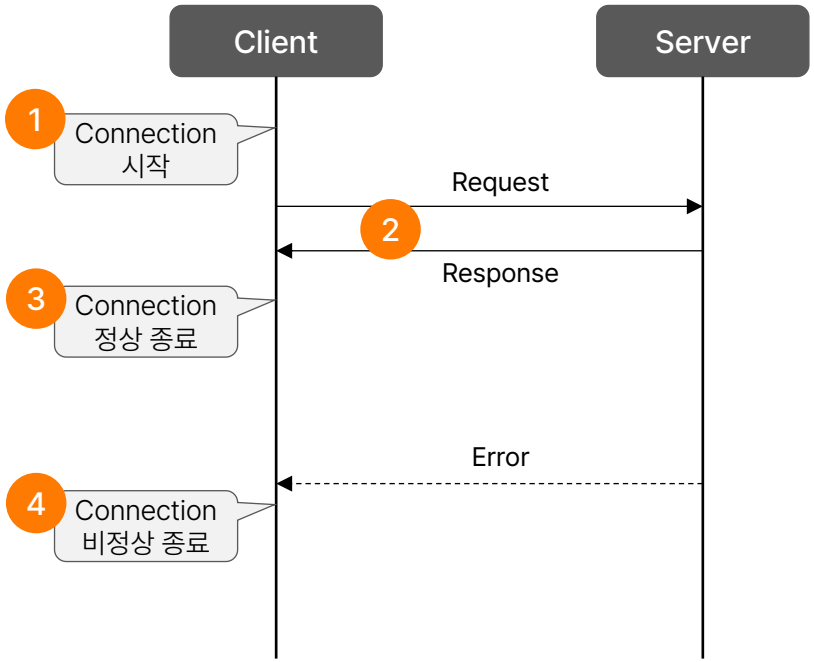
```
{"data": "안녕하세요"}
{"data": {"name": "search_manual"}}
{"data": {"error": "timeout"}}
```

좋은 예시 :

```
{"type": "token", "content": "안녕하세요"}
{"type": "tool_call_start", "tool_name": "search_manual", "args": {...}}
{"type": "tool_call_end", "tool_name": "search_manual", "results": {...}}
{"type": "done", "finish_reason": "stop"}
{"type": "error", "code": "timeout", "message": "..."}
```

1-(3). Lifecycle

언제 streaming이 시작되고 끝나는가?



단계별 결정 사항 :

1. Connection 시작
   - 초기 메타데이터 전송 여부 (입력 토큰 수, model info, etc.)
   - **Stream ID 발급** (디버깅/로깅용)

2. Active
   - 이벤트 간 최대 간격 (Idle timeout)

   A Keep-alive 매커니즘 : 장시간 tool call 중 끊김 방지

3. 정상 종료
   - 명시적 “done” 이벤트 vs HTTP connection close 만으로 종료
   - 종료 시 cleanup 이벤트 처리 (최종 토큰 수, latency 등)

4. 비정상 종료
   - 서버 에러 → “error” 이벤트 후 종료? 또는 HTTP error?

   B Client disconnect → 서버에서 어떻게 인지하고 처리?

   C Agent 중간 작업 중 disconnect → tool call 처음부터 진행? 중단?

1-(3A). 장시간 Tool Call 중 Stream 유지

Stream이 끊기지 않는 것의 핵심은 '계속 무언가가 흐르고 있다'는 신호를 주는 것 !!!

문제 상황 :

- Agent tool call 시작 (e.g., DB 조회, 외부 API 호출 등)
- Tool 응답까지 30초 이상 소요

- 30초 동안 Stream에 아무런 이벤트도 흐르지 않음

- 일부 proxy/load balancer가 idle stream 끊어버림
- 또는 사용자가 "멈춤"으로 인식

대응 방안 :

1. Tool call 시작 /종료 이벤트로 **keep-alive** :
   - "tool_call_start" 이벤트 발생 → 사용자에게 "도구 사용 중" 표시
   - Stream 자체가 한번 활성화 되도록 함

2. Heartbeat/Ping 이벤트 :
   - 일정 간격으로 ping 또는 empty comment 전송
   - SSE 주석 라인(" : keepalive")으로 전송

   ← 매우 긴 작업(30초 이상)을 다룬다면 명시적으로 Heartbeat 적용

3. 중간 진행 이벤트 :
   - Tool 이 진행 중이라는 **메타 이벤트 전송**
   - (e.g.) "검색 중... (XX초 경과)"

1-(3B). Client Disconnect 처리

사용자가 떠나면... Disconnect 처리 필요

시나리오 :

- 사용자가 응답을 기다리다가 페이지를 닫아 버림
- 새로운 대화창을 열고, 새 메시지를 보냄
- 또는 네트워크 끊김

Server 대응 방안 :

1. Disconnect 감지 : HTTP connection이 끊기면 Server 인지

2. 진행 중 작업 처리 :
   - LLM 호출 중 → 작업 취소 처리? Retry 통해 끝까지 진행?
   - Tool Call 중 → 도구 호출 취소 처리? Retry 통해 끝까지 진행?

3. State 처리 :
   - 부분 응답을 thread state에 저장할 것인가?
   - 사용자가 돌아왔을 때 대화 이어가기가 가능하도록 할 것인가?

Trade-Off :

- Disconnect 감지 시 작업 취소 처리
  - 장점 : 리소스 절약, 비용 절감
  - 단점 : 부분 응답 손실, 재시도 시 처음부터 진행 필요
- Retry 통해 끝까지 진행
  - 장점 : 사용자가 돌아왔을 때 대화를 이어갈 수 있음
  - 단점 : 리소스 소비, 확인하지 않을 답변까지 생성
- 부분 응답 저장 후 작업 취소 처리
  - 장점 : 리소스 절약, 깔끔한 프로세스 처리
  - 단점 : 구현 복잡도

1-(3ⓒ). Backpressure와 흐름 제어

Client가 Server 처리를 따라오지 못할 때...

시나리오 :

- Server가 매우 빠르게 이벤트 생성
- Client가 처리/랜더링을 따라오지 못함
- 버퍼가 쌓이거나 메모리 압박

일반적인 대응 방안 :

1. TCP 자체 backpressure :
   - HTTP/TCP가 자체적으로 흐름 제어
   - Client Buffer가 차면 서버 송신이 자연스럽게 느려짐
   - 대부분의 SSE 시나리오에서 충분

2. 명시적인 ack 패턴 적용 :
   - Client가 처리 완료 신호를 서버로 전송
   - Server가 ack 받은 후 다음 이벤트 전송

3. 이벤트 throttling :
   - 서버가 의도적으로 이벤트 rate 제한
   - (e.g.) token 100개씩 묶어서 전송

2. Frontend, Framework 비교

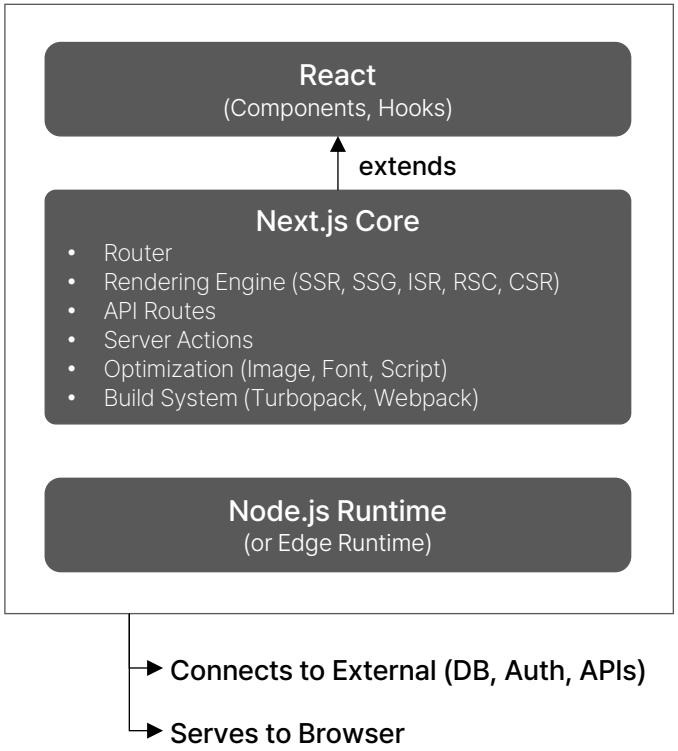
AI Agent Production 관점

구분		Streamlit	React	Next.js
내용		• Python-based Web App Framework	• JavaScript UI Library	• React-based Full Stack Framework
특징		• 파이썬 데이터 처리와 통합 용이 • UI 커스터마이징 제한	• UI Component 재사용성 우수 • 라우팅, SSR, Build 등 직접 구성 필요	• 풀스택 환경 통합, 다양한 렌더링 모드 제공 • 일부 기능은 Vercel 배포 시 가장 매끄러움
AI Agent Product	**Streaming UX 지원**	• `st.write_stream` 등으로 토큰 스트리밍 기본 지원	• Vercel AI SDK • 또는 Fetch + ReadableStream으로 구현	• Vercel AI SDK + Route Handler로 서버/클라이언트 일관 구성
	Multi-Event Streaming	• `st.status`, `st.fragment`로 구현 가능 • 표현 자유도 제한적	• AI SDK message parts • 또는 자체 상태 관리로 구현	• AI SDK의 message parts 활용
	Tool Call 가시화	• 컨테이너 조합으로 구현 가능 • 표준 패턴 없음	• AI SDK tool parts 활용 가능 • UI는 직접 구성	• AI SDK tool parts 활용 가능 • UI는 직접 구성
	사용자 인증 & 세션 관리	• `st.login` 기본 지원 • 세밀한 권한 관리는 제한적	• 외부 라이브러리 활용 (Auth0, Clerk, Firebase Auth 등)	• 외부 라이브러리(Auth.js 등) + Middleware로 서버측 보호
	동시성/확장성	• 세션별 독립 실행, 세션당 메모리 부담 • 수평 확장 시 sticky session 필요	• 정적 배포로 Frontend 확장 용이 • 전체 확장성은 Backend에 의존	• 서버 기능 포함 • 확장성은 배포 인프라에 의존
	상태 관리 (thread, session)	• Session State API (세션 단위, 영속화는 별도 구현)	• 외부 상태 관리 도구 (Redux, Zustand 등)	• React 상태 관리 + Server Components / Server Actions 결합 가능
	Product 적합도	• PoC, Demo 적합 • 외부 고객용 제품 UI에는 제약	• UI 자유도 높음 • Backend, Routing 별도 구성 필요	• Full Stack 제품 구성 시 • 에이전트 사용 시 별도 Backend 필요

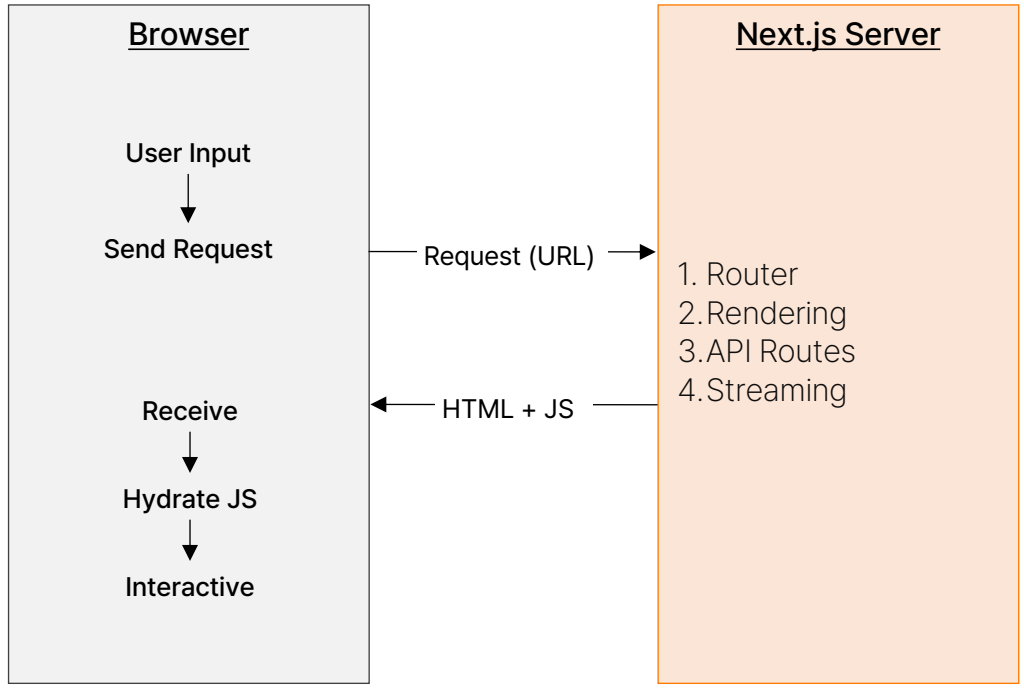
2-(1). Next.js

- Full-stack Web Application 구축 위한 React Framework
- React 위에 Production 레벨의 Web App을 만들기 위한 표준 환경을 묶어둔 것

Next.js Framework :



Request Flow :

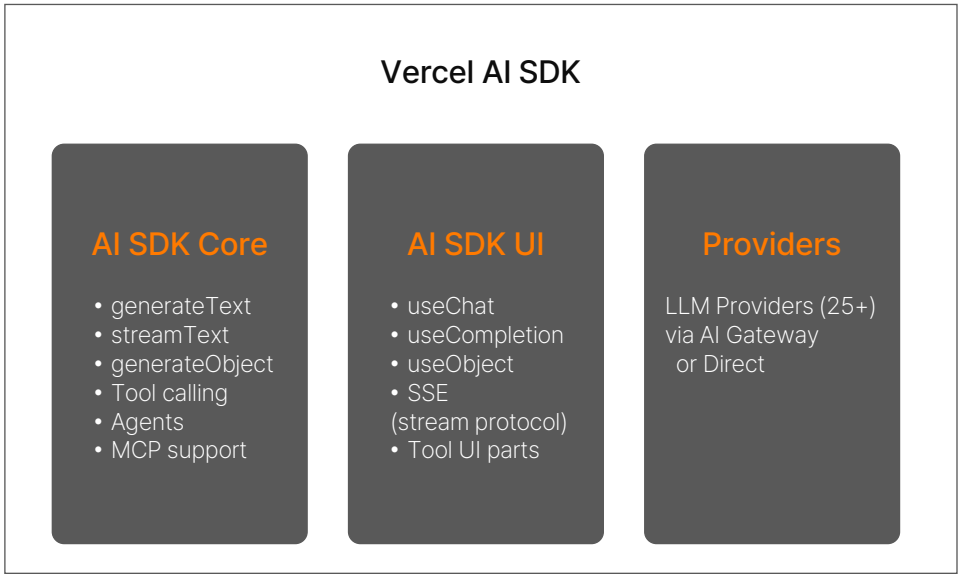


※ Rendering
- SSR : Render HTML per Request
- SSG : Serve pre-built HTML
- ISR : Serve cached + Regenerate
- RSC : Server-render Components

2-(2). Vercel AI SDK

AI Application 구축 위한 TypeScript Toolkit

- TypeScript 기반의 AI Application 개발 도구
- LLM 호출, Streaming, Tool calling, Chat UI 등 AI Application에 필요한 공통 패턴을 추상화 함



- AI SDK Core (Server-side) : LLM 호출 통합
- AI SDK UI (Client-side) : Chat UI 구현
- Providers : OpenAI, Anthropic, Google 등 (25+ provider 통합)

주요 특징 :

- Provider 추상화

```
// OpenAI → Anthropic 전환이 모델 문자열만 변경

generateText({ model: 'openai/gpt-5', prompt: '...' });
generateText({ model: 'anthropic/claude-opus-4', prompt: '...' });
```

- Streaming 표준화
  - SSE 기반의 표준 streaming protocol 제공
  - Token, tool call, error 등 multi-event를 표준 포맷으로 처리

- Frontend Hook
  - Hook : useChat, useCompletion, useObject 등
  - Hook으로 Chat UI의 공통 패턴 처리

- **Tool Calling 통합** : Multi provider의 tool calling 기능을 API로 추상화
- **Type Safety 확보** : TypeScript 기반으로 Front-Back 간 type 일관성 확보
- **Agent 및 MCP 지원** : AI SDK 6 부터 지원

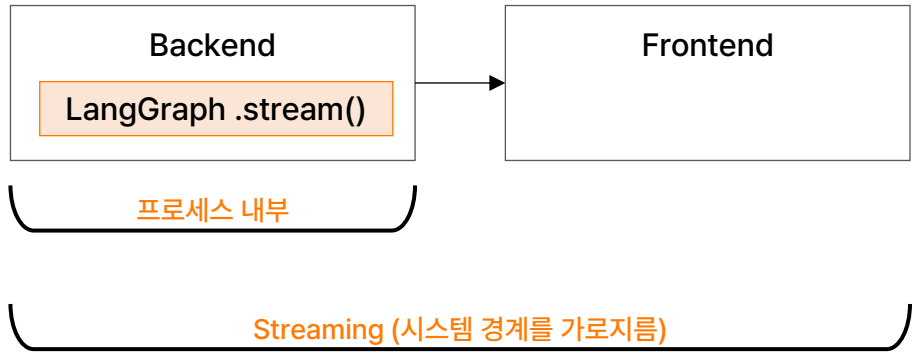
3. Streaming

SUMMARY

SUMMARY

Stream vs Streaming

용어 정리



구분	LangGraph .stream() / .astream()	Streaming
개요	LangGraph, Graph 객체의 메서드	시스템 간 점진적 데이터 전달 패턴
경계	단일 프로세스 내부	프로세스 / 머신 / 네트워크를 가로지름
전달 채널	Python async iterator	HTTP 기반 프로토콜 (SSE / WebSocket / chunked)
데이터 단위	Graph node 실행 결과	의미 있는 이벤트 (token, tool call, planning, error 등)
수신자	같은 프로세스 안의 호출자 (Python 코드)	다른 시스템 (브라우저, Client)
Streaming 관점	Streaming Source	Delivery infrastructure

SUMMARY

1. Streaming 위한 3가지 차원

3가지 독립적인 결정의 조합

Streaming

1 Transport

어떻게 streaming 채널을 만드는가?

- HTTP Streaming (chunked transfer)
- Server-Sent Events (SSE)
- WebSocket
- gRPC streaming

2 Content

무엇을 streaming할 것인가?

- Text token : LLM 응답 글자
- Structured event : tool call 시작, 종료, 결과
- State delta : agent state 변화
- Custom payload : Product 특화 이벤트

3 Lifecycle

언제 streaming이 시작되고 끝나는가?

- 요청 시작을 Streaming 시작으로
- 종료 신호 처리
- 에러 시 처리
- Client 끊김 시 처리

Hands-On

Notion > Streaming

- ❑ Notion 가이드를 확인하고,
- ❑ Frontend로 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 해당 내용을 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인합니다.

모듈 연결도

Streaming

※ transport.ts : 전달받은 SSE을 열어 아이템을 꺼내
아이템 종류에 따라 화면에 반영

Presentation Layer

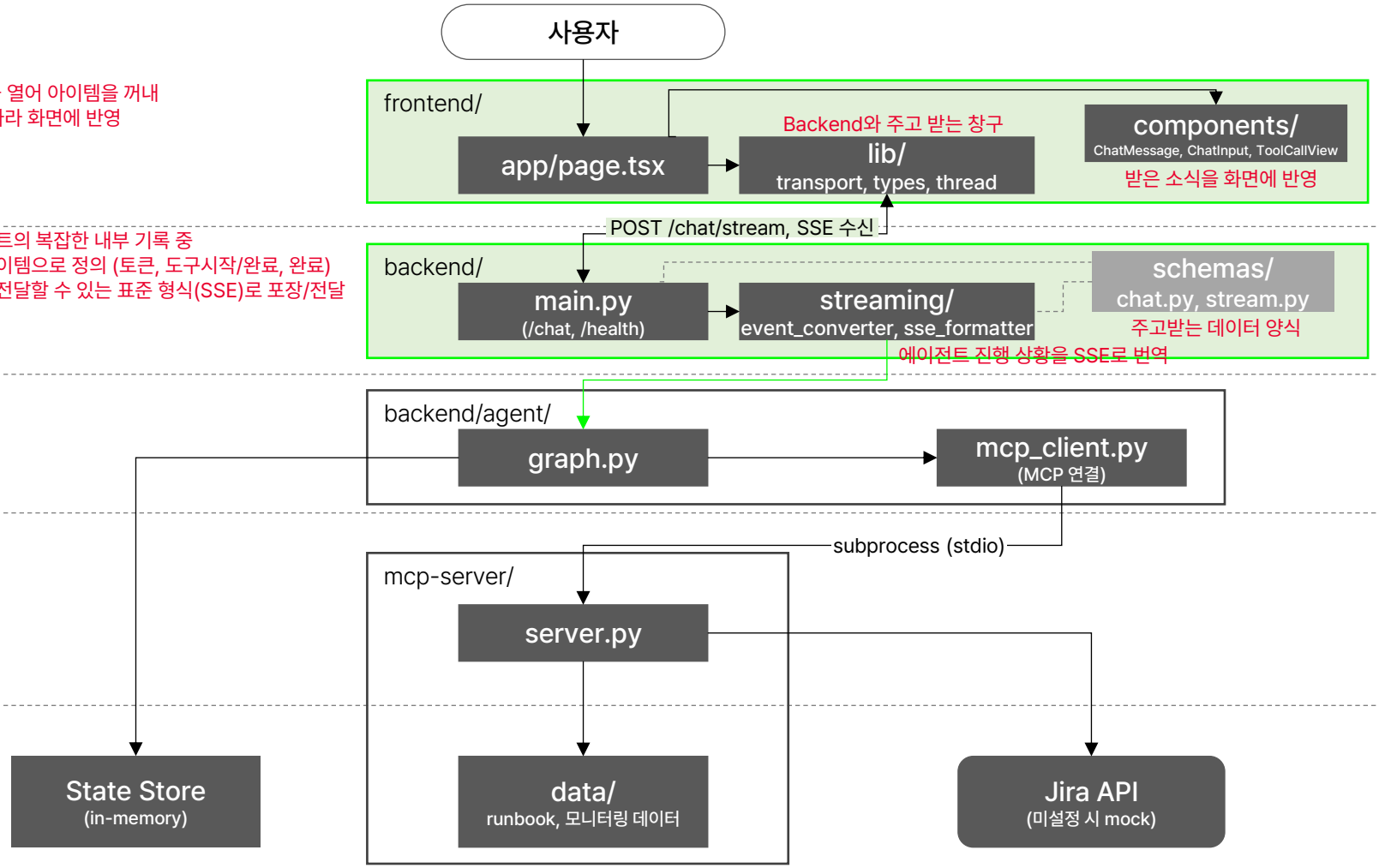
※ event_converter.py : 에이전트의 복잡한 내부 기록 중
화면에 필요한 것만 골라서 아이템으로 정의 (토큰, 도구시작/완료, 완료)
※ sse_formatter.py : 아이템을 전달할 수 있는 표준 형식(SSE)로 포장/전달

Application Layer

Orchestration Layer

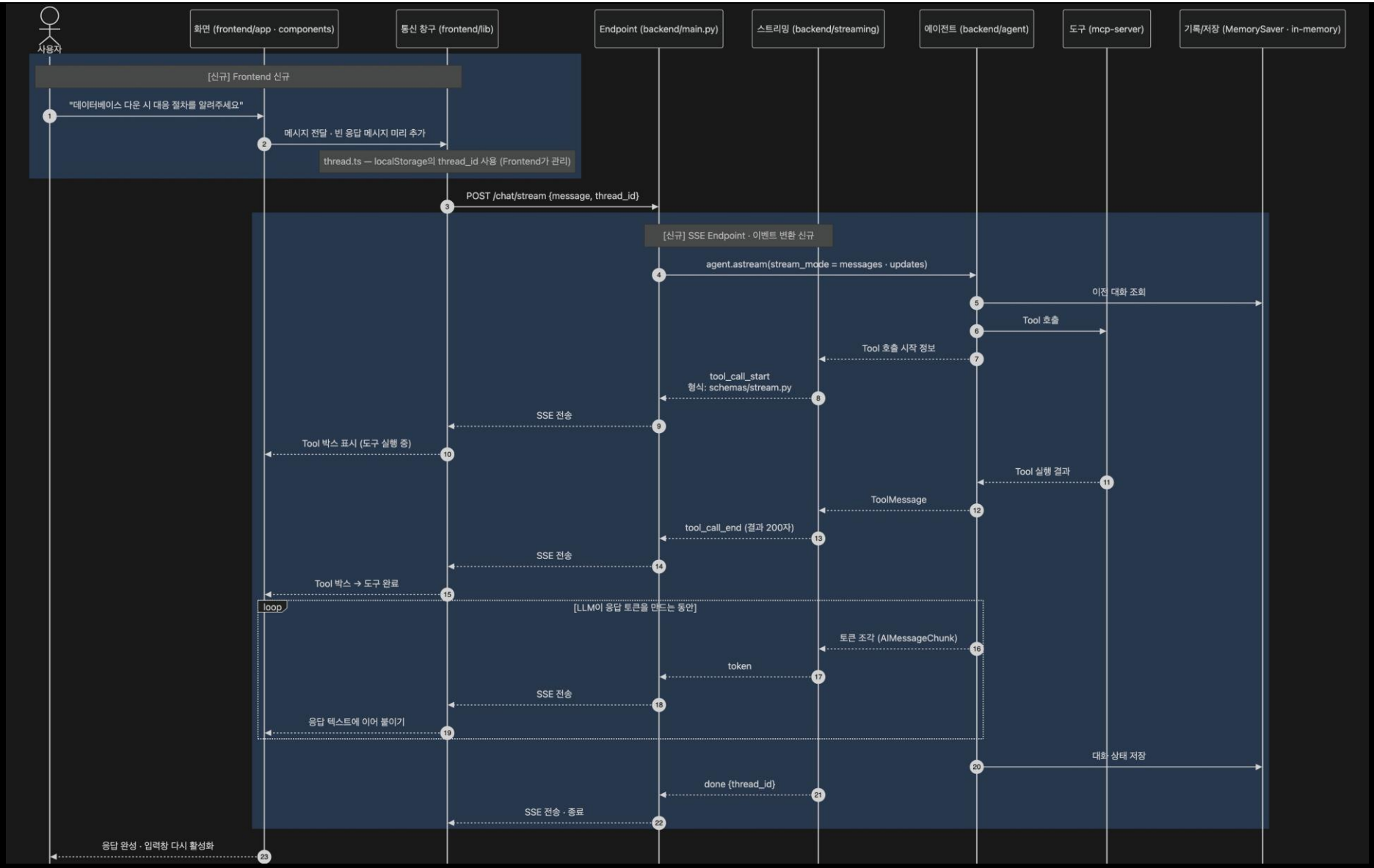
Tool Layer

Data Layer



서비스 흐름도

Streaming



Hands-On

Notion > Multi-Agent Streaming

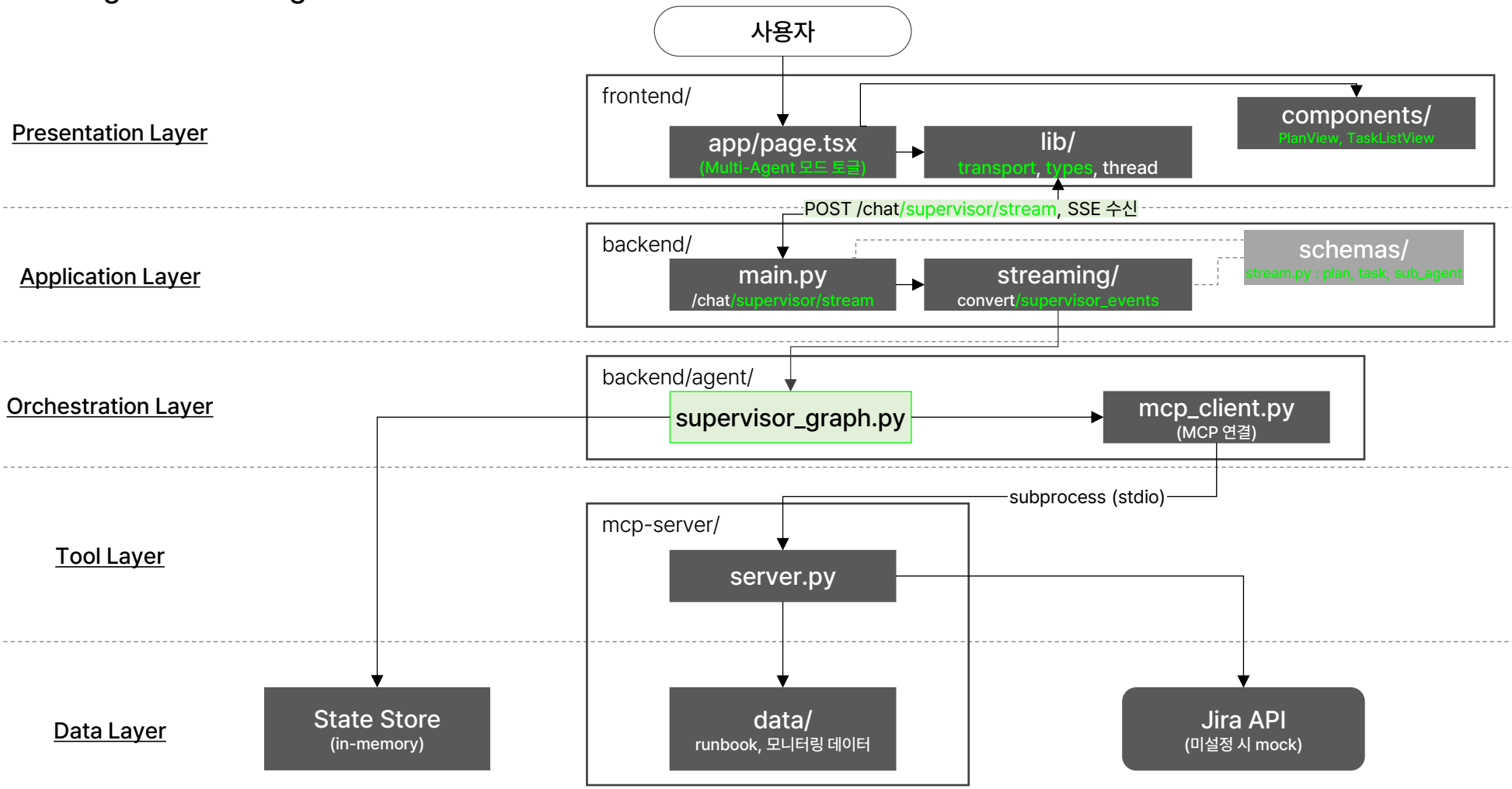
- ❑ Notion 가이드를 확인하고,
- ❑ Frontend로 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 해당 내용을 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인 합니다.

Hands-On

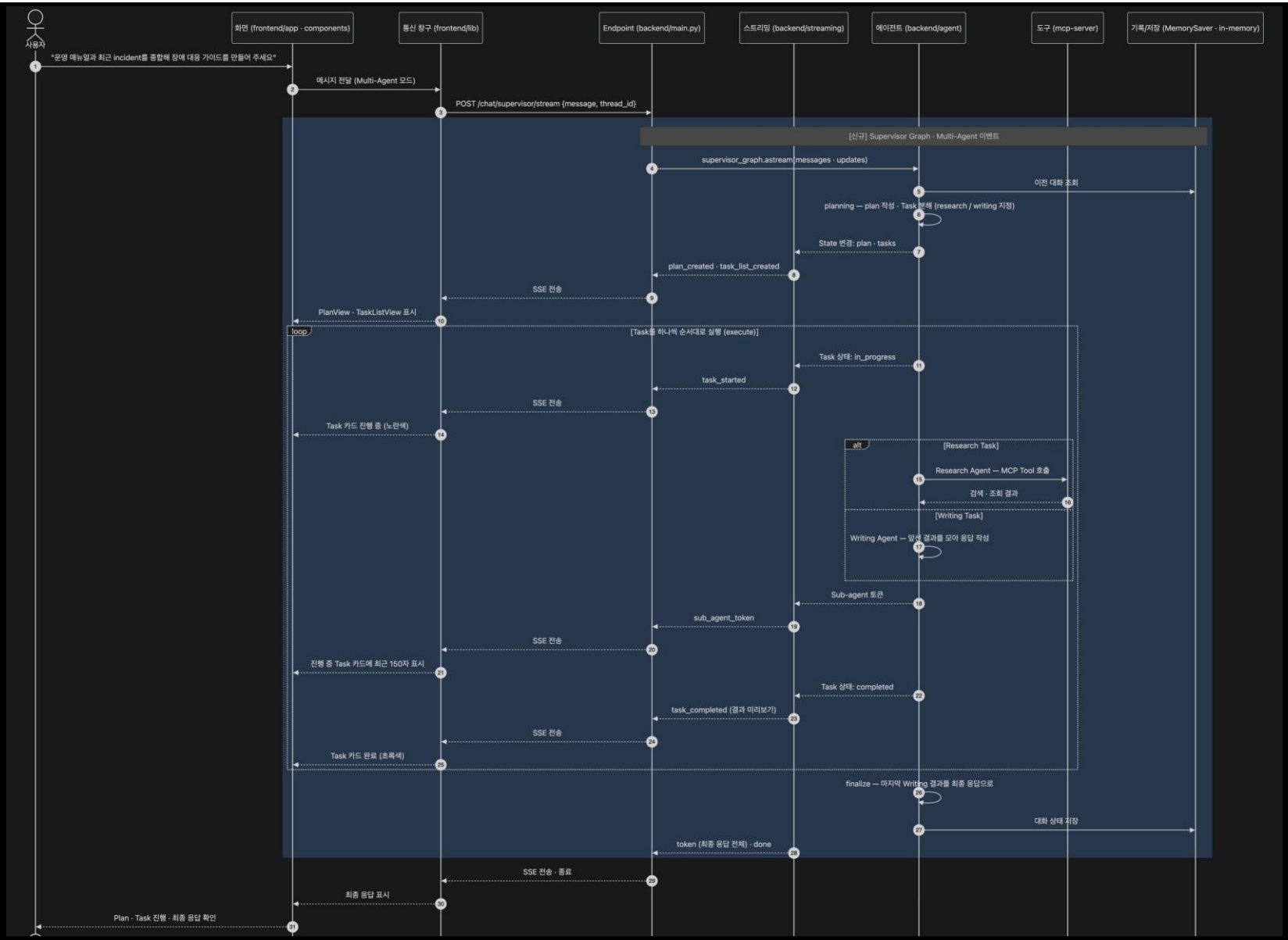
모듈 연결도

Multi-Agent Streaming



서비스 흐름도

Multi-Agent Streaming



AI Campus 생성형AI

4. 통합 환경의 운영 All

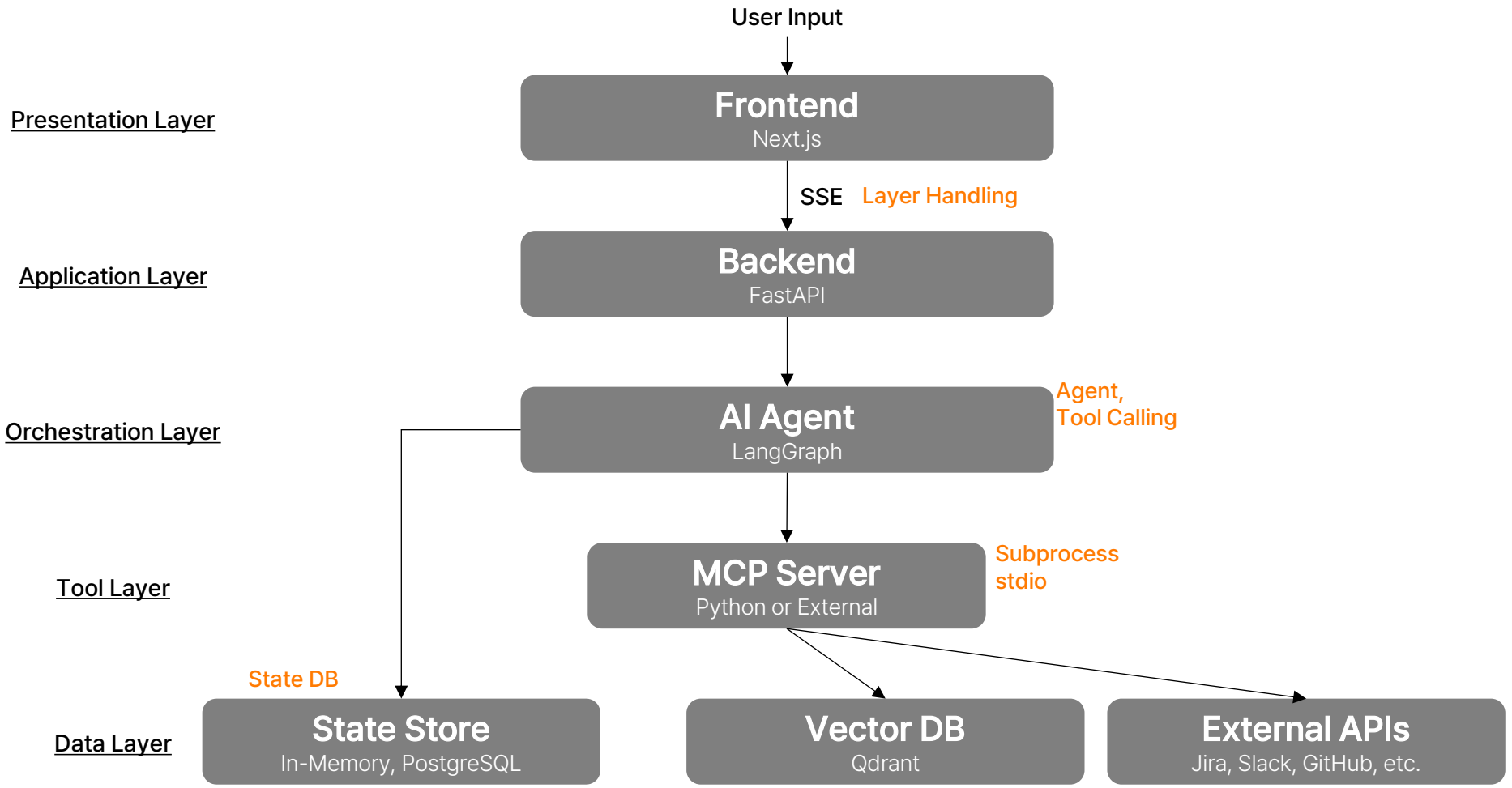
- Observability & Eval
- Error Handling
- Cost & Latency 관리
- Stateless Session 전환

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

System Diagram

for Capstone



Callback

DONE

- LangGraph checkpointer(MemorySaver) + LangSmith Tracing
- Agent Eval : Dataset, Evaluator, Score
- 현재까지 통합한 환경 : Single instance, In-memory state

"통합 환경의 운영 문제를 도구로 진단/해결"

YET

- Observability로 통합 환경 tracing
- 선행 Eval 아이템을 product에 연결
- Layer간 Error 책임 설계 (+ Graceful Degradation)
- 통합 환경의 Cost & Latency 측정 (+ 최적화)
- Single instance Stateful → Multi-instance Stateless로 전환

1-(1). Observability

용어의 이해

Observability, 관찰 가능성 :

- The ability to understand a system's internal state by analyzing its external outputs
- 시스템의 외부 출력을 분석하여 복잡한 내부 상태를 이해 및 추론하여, 장애의 근본 원인을 선제적으로 파악하는 능력
- 시스템의 내부 상태에 대한 의미 있는 결론에 도달하기 위해 출력을 평가하는 방법
  - Monitoring : 미리 정의된 임계치를 기반으로 장애 발생 여부를 알림
  - Observability : 데이터 연결성 통해 미처 예상하지 못한 문제까지 원인을 파악하고자 함 ← "Unknown"을 탐지하려면 이 또한 사전에 정의가 되어야 함
    (+) 이를 파악하기 위해 어떤 차원의 데이터를 수집할지도 사전에 결정 필요

- 핵심 구성 요소 :
  - Metrics : 시간 경과에 따른 시스템 성능 데이터 (CPU 사용량, 메모리 등) ← 해당 데이터들이 별도로 수집 및 저장되면 상관/연관 관계를 파악하기 어려움
  - Logs : 특정 이벤트 발생 시 기록되는 상세한 기록
  - Traces : 분산 시스템 간의 요청 흐름과 처리 과정 추적 ← Log와 Tracing 정보는 트래픽에 비례하여 저장 비용 발생

- 참고 :
  - Observability : 시스템이 무엇을 하고 있는지 관측 - descriptive
  - Eval : 시스템 출력이 얼마나 기준에 부합하는지 평가 – judgmental based guide

1-(1). Observability

AI Agent 관점

- 비결정적 LLM : 같은 입력에 다른 출력
- Multi-Agent 구성 시 여러 번 LLM 호출 + tool call + state 변화가 발생하여 로그 만으로는 충분히 파악하기 어려움
- Token 사용량을 추적하여 가시화 해야 함

- 구성 요소 :

구분	내용
Trace	• 하나의 요청에 대한 전체 실행 경로
Span	• Trace의 한 단계 • 개별 LLM 호출, Tool call 등
Metrics	• 정량적인 평가 • Latency, token usage, cost, etc.
Metadata	• Tag, User info, Version 등 분류 정보 • (e.g.) thread_id, user_id, agent_version, etc.

Observability가 답하는 질문 (예시) :

- 이 요청은 왜 30초나 걸렸나요?
  → Trace에서 병목 단계 식별
- 이번 달 비용이 왜 증가했나요?
  → Token 사용량 패턴 분석
- 오류가 어디서 발생했나요?
  → Trace에서 실패 span 추적
- Plan은 어떻게 세워졌나요?
  → Planning의 prompt와 completion 확인

1-(2). Eval

시스템 출력이 얼마나 기준에 부합하는지 평가

- 시스템의 출력을 외부 기준과 비교하여 점수 또는 판단을 부여하는 활동
- AI Agent에서는 "이 응답이 사용자에게 좋은 응답인가"를 측정

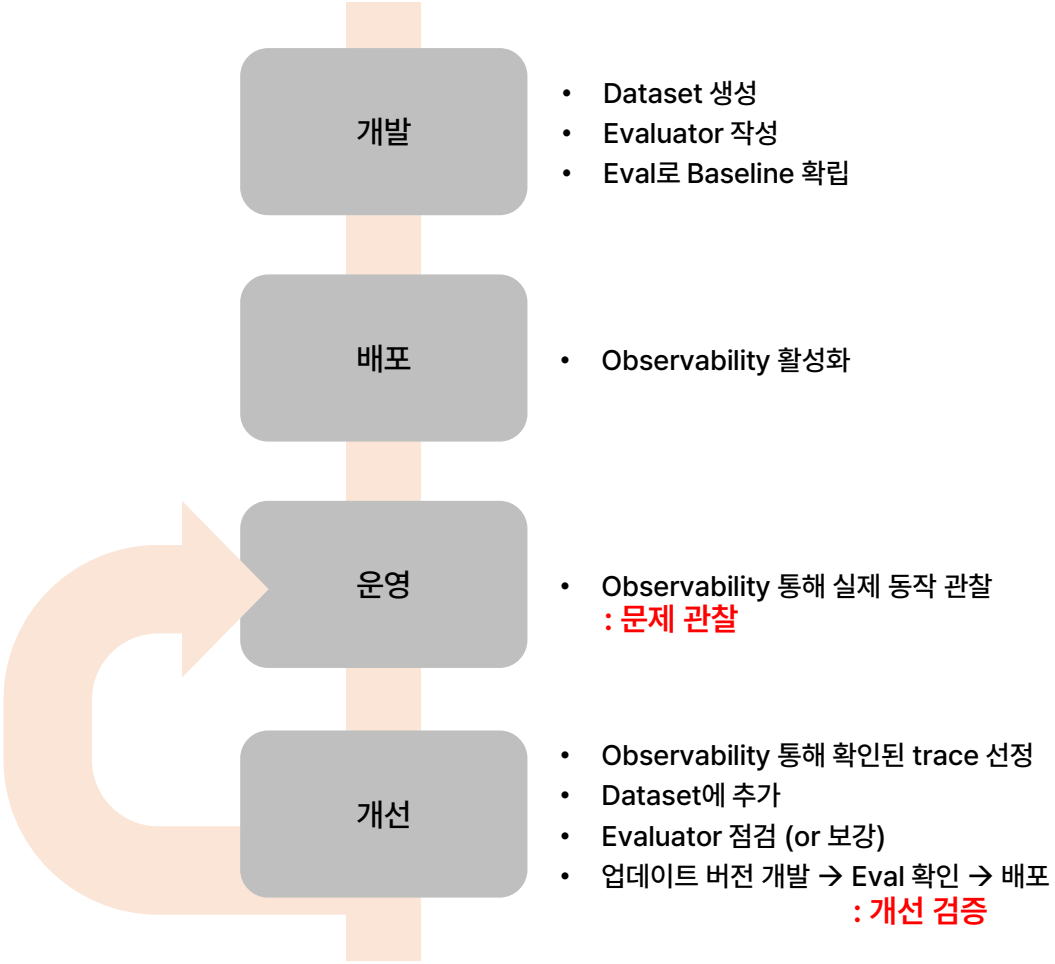
- 구성 요소 :

구분	내용
Dataset	• 질의 + (기대)응답
Target	• 평가 대상 : Agent, Function, System 등
Evaluator	• 출력에 점수를 부여하는 함수
Score	• 평가 점수
Experiment	• Target * Dataset의 한 번의 실행 결과 전체

- Reference-based : 정답과 비교
- LLM-as-a-Judge : LLM + Rubric 기반으로 평가 (응답 품질)
- Heuristic : 규칙 기반 검증 (형식 검증)

- Eval 질문 (예시) :
  - 이번 프롬프트 변경이 응답 품질을 개선했나요? → 같은 dataset으로 프롬프트 버전(v1, v2)별 실행 결과 비교
  - Multi-agent가 Single agent 대비 답변 결과가 나아졌나요? → 같은 dataset으로 시스템별 실행 결과 비교
  - LLM 모델(+버전)을 변경해도 품질은 유지되나요? → 같은 dataset으로 모델별 실행 결과 비교

1-(3). Observability & Eval



- Dataset 생성
- Evaluator 작성
- Eval로 Baseline 확립

- Observability 활성화

- Observability 통해 실제 동작 관찰
  : 문제 관찰

- Observability 통해 확인된 trace 선정
- Dataset에 추가
- Evaluator 점검 (or 보강)
- 업데이트 버전 개발 → Eval 확인 → 배포
  : 개선 검증

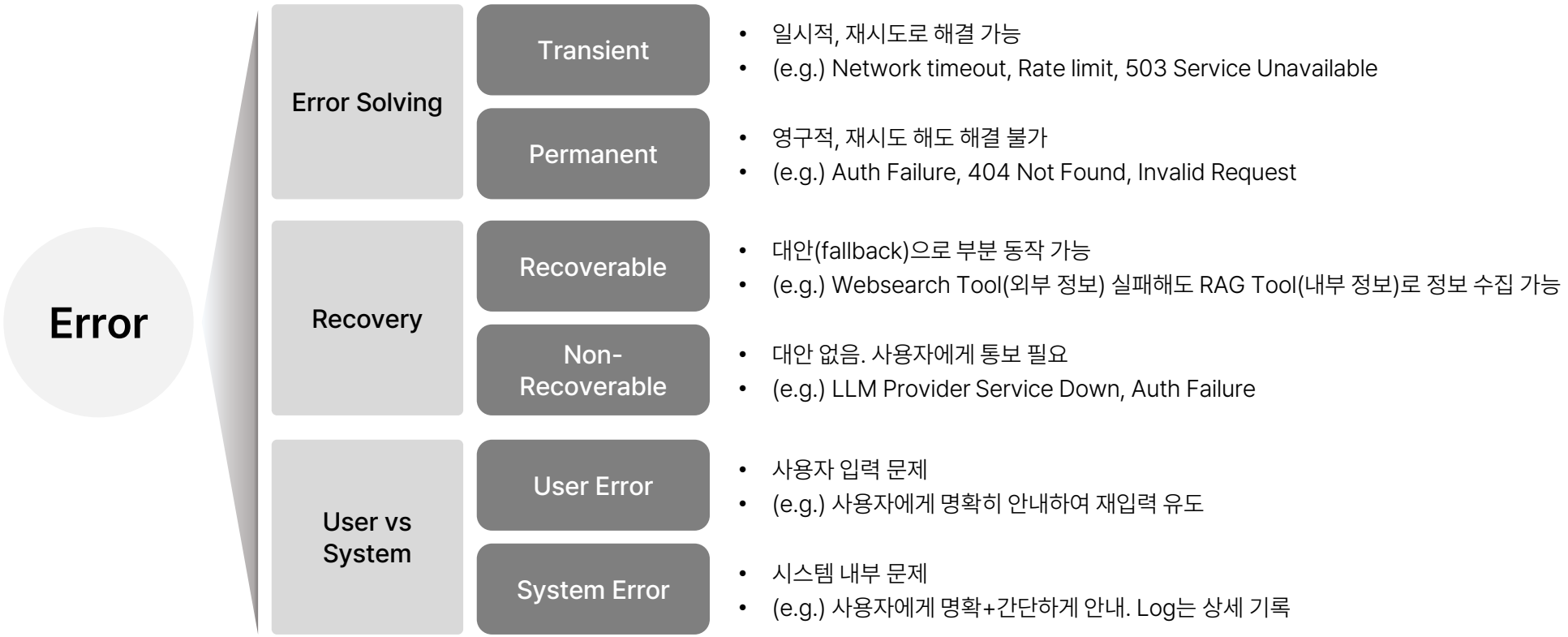
❑ **Observability와 Eval은 분리될 수 없는 한 쌍 입니다.**

- Observability만 있으면 변경이 좋은지 확신 할 수 없음
- Eval만 있으면 운영의 실제 문제를 대응 할 수 없음

➔ 두 가지가 함께 동작할 때, AI Agent Product은 점진적으로 개선됨

2. Error

Error 분류



- 일시적, 재시도로 해결 가능
- (e.g.) Network timeout, Rate limit, 503 Service Unavailable

- 영구적, 재시도 해도 해결 불가
- (e.g.) Auth Failure, 404 Not Found, Invalid Request

- 대안(fallback)으로 부분 동작 가능
- (e.g.) Websearch Tool(외부 정보) 실패해도 RAG Tool(내부 정보)로 정보 수집 가능

- 대안 없음. 사용자에게 통보 필요
- (e.g.) LLM Provider Service Down, Auth Failure

- 사용자 입력 문제
- (e.g.) 사용자에게 명확히 안내하여 재입력 유도

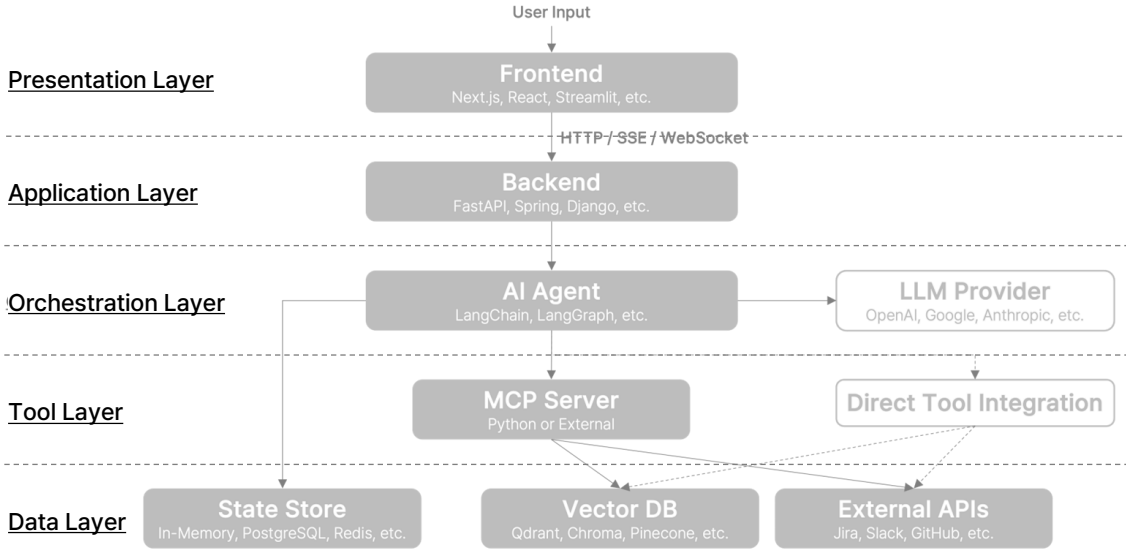
- 시스템 내부 문제
- (e.g.) 사용자에게 명확+간단하게 안내. Log는 상세 기록

Error를 모두 같은 방식으로 처리하면 안 됩니다.
각 종류에 맞는 전략 필요 !!!

2-(1). Error Responsibility

- 하위 Layer → 상위 Layer : 무엇이 잘못 되었는지 명확하게 전달
- 상위 Layer : 전달받은 Error를 현재 Context에서 판단
- 최상위 Layer (Frontend) : 사용자에게 전달되는 메시지 표현 책임

Layer :



Error Responsibility :

- 사용자에게 명확하게 전달
- Retry UX, Disconnection 처리 등

- HTTP 응답 형식화 및 사용자 친화적 형태로 변환
- Stream 중간 에러 처리

- Sub-agent 또는 Tool 실패 시 fallback 결정
- Partial result로 사용자에게 전달 가치 판단

- MCP Server와의 통신 관리 : Connection 복구, Timeout 처리
- 실패 시 Tool 단위 실패 내용을 상위 전달

- 외부 시스템 통신 Error 발생 → Retry 1차 시도
- 자체 해결 불가 시 Error Type 상위 전달

2-(2). Retry 정책

- 대상 : Transient Error만. Permanent Error는 바로 실패 처리
- 횟수 : 상한을 명시해야만 함
- 시간 : 대기 시간을 점점 늘림 (0.5초 → 1.5초)
  외부 시스템이 회복할 시간을 주고, 요청이 한꺼번에 몰리는 것을 막기 위함
- 처리 : 그럼에도 끝내 실패하면, 표준 에러로 바꿔 상위 Layer에 전달

- 주의할 점 :
  - Action Tool을 재시도하면 중복이 생길 수 있음
    (e.g.) `create_jira_ticket`을 재시도하면 티켓이 여러 건 만들어질 수 있음
  - 여러 레이어가 각자 재시도하면 회수가 곱해짐
    (e.g.) Data Layer에서 3회, Tool Layer에서 다시 3회를 하면 최대 9회 Retry 발생하게 됨

- Trade-off : 성공률은 올라가지만, 그만큼 응답이 늦어지게 됨

2-(3). Graceful Degradation

Degradation :

- [사전] 저하, 하락, 악화, 분해
- 무언가의 품질, 가치, 상태가 시간이 지나거나 외부 요인으로 인해 나빠지거나 깎이는 과정
- 다양한 분야에서 가치가 떨어지는 현상을 지칭할 때 주로 사용됨

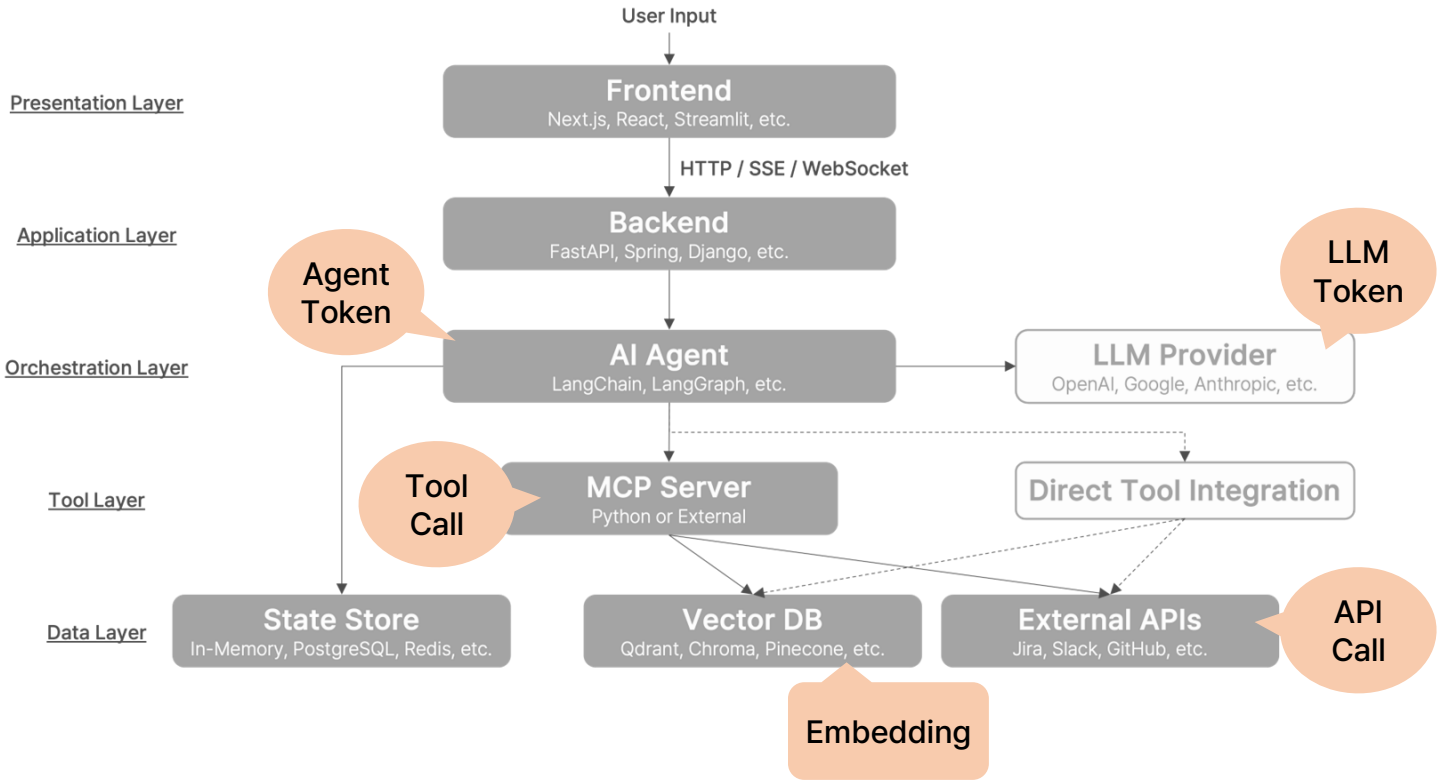
Graceful Degradation :

- 시스템의 일부가 실패해도...
- 남은 부분으로 사용자에게 최대한의 가치를 전달하는 설계 패턴

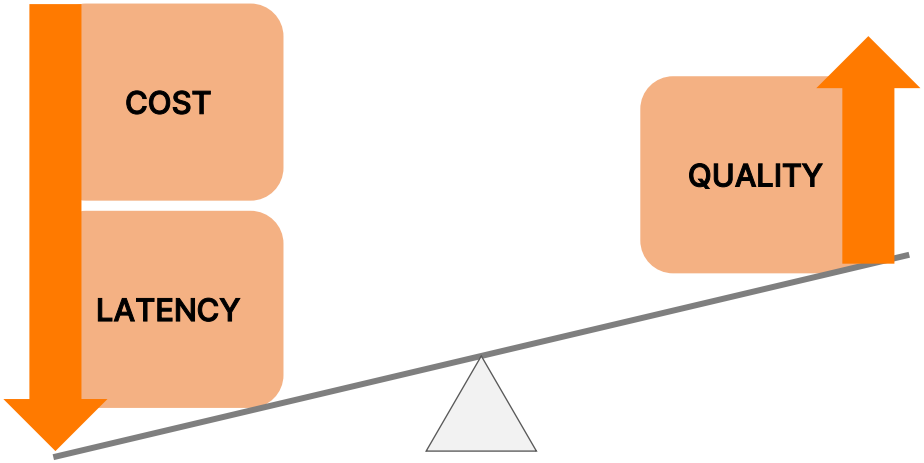
- Don't : 시스템이 동작하거나 / 동작하지 않거나 ← Binary
- Do : 시스템이 동작 / 부분 동작 / 제한된 동작 / 동작 불가 ← 여러 상태 존재. 각 상태에서 사용자에게 적절한 경험 제공

3. Cost & Latency

- Cost의 대부분은 LLM token 비용($/request or token)으로 서비스 품질에 직접적으로 관련
- Latency도 LLM 호출과 관련하여 발생 (sec/request)



3-(1). Cost & Latency vs Quality



Agent - 통합시스템에서의 특수성 :

- 호출이 누적됨 :
  - Single agent : LLM 호출 1~3회 수준
  - Multi-agent : LLM 호출 (최소) 5+ 수준
  - Caching : Cost ↓, Latency ↓ → Quality ↑ (일관성 측면)
- Tool 호출이 LLM 호출을 추가적으로 요구함 :
  - Tool call → Tool 응답 → 그 응답을 처리하는 LLM 호출
  - 하나의 tool 호출이 LLM 호출 2번 발생
  - Tool 호출 감소 : Cost ↓, Latency ↓ → Quality ↓ (정보 부족)
- Context 누적 :
  - LLM 호출이 이전 메세지를 모두 포함
  - Multi-turn이 길어질수록 input token 증가
  - Context 압축 : Cost ↓, Latency ↓ → Quality ↓
- Latency 누적 :
  - 순차 실행 시 latency는 각 단계 latency의 총합
  - 한 단계만 느려져도 전체 서비스 지연과 연결
- Sub-agent 효율성 :
  - 전체 응답만 보면 sub-agent 비효율성을 파악하기 어려움 → Observability
  - 병렬 실행 : Latency ↓ → Cost 변화 거의 없음

[참고] 병렬 실행이 가능한 조건

병렬 실행 충족 조건 (4가지 모두 충족) :

서로의 결과를 입력으로 사용하지 않음	같은 State를 동시에 업데이트 하지 않음	실행 순서가 결과에 영향 주지 않음	하나가 실패해도 나머지는 계속 진행 가능
A 에이전트가 B 에이전트 결과를 기다리지 않음	각 Task는 자기 결과만 반환하고, 합치는 일은 별도 노드에서 진행함	생성 및 수정 같은 Action은 주의!!! (조회는 상관 없음)	최종 실행 결과에 일부 실행의 실패에 대해 에러 내역으로 표시해도 됨

Trade-off :

- 장점 : Latency가 줄고, Cost는 거의 그대로
- 단점 :
  - 전체 대기 시간은 가장 느린 task에 맞춰짐
  - 동시 호출이 늘어 Rate limit에 걸릴 위험이 커짐

4. Session

- Stateful은 단일 인스턴스에서 State 손실 허용 가능성을 인지하고 있는 환경에서는 충분히 동작 가능
- Stateless의 이점은 Multi-instance 배포가 필요할 때 드러남 → Product AI Agent에서 요구되는 부분

```
# 현재 코드

from langgraph.checkpoint.memory import MemorySaver

checkpointer = MemorySaver()
```

MemorySaver 동작 방식 : ➔ **Stateful**

- Thread state를 backend 프로세스의 메모리(RAM)에 저장
- Python dict 형태로 보관
- Backend 프로세스 종료 시 모든 state 손실

Stateless Application :

- State를 외부 Store (DB, Cache 등)에 위탁
- 인스턴스 자체에는 State를 보관하지 않음
- State와 인스턴스가 분리되어 있음

Stateful Application :

- State를 자체 메모리(RAM)에 보관
- 인스턴스 프로세스가 종료되면 state 손실
- State와 인스턴스가 강하게 결합되어 있는 구조

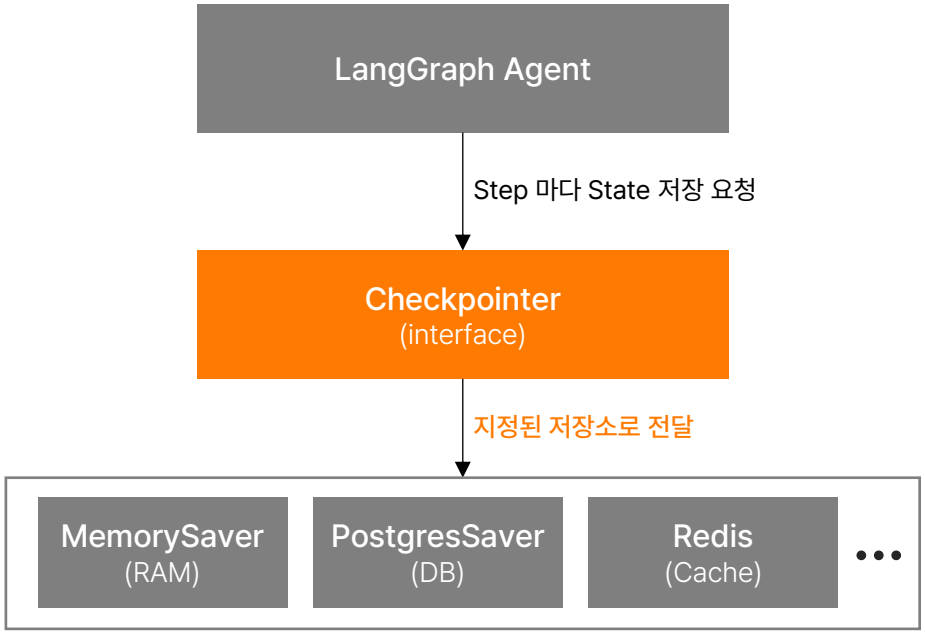
[Remind] thread_id vs checkpoint_id

- **세션 식별자 – thread_id :**
  - 발행 주체 : **Client(Front or Back)가 생성**하여 요청 시 주입함
  - 담당 역할 : 대화의 '방 번호' 또는 '세션 ID' 역할 담당. **동일한 유저와의 연속된 대화 Context를 묶어주는 단위**
  - 관리 방식 : Client가 쿠키, JWT 토큰, 혹은 로컬 스토리지 등에 보관하다가
    LangGraph 에이전트를 호출할 때 config={"configurable": {"thread_id": "abc-123"}} 형태로 넘겨줌

- **스냅샷 식별자 – checkpoint_id :**
  - 발행 주체 : **LangGraph 라이브러리(엔진) 내부에서** 자동으로 발행
  - 담당 역할 : **에이전트 내의 Node가 실행을 마칠 때마다**, 해당 시점의 상태를 versioning 하는 Git 커밋 해시 같은 역할을 함
  - 관리 방식 : LangGraph 내부적으로 uuid 등을 이용해 생성하여
    Checkpointer 통해 메모리/DB/캐시 등에 적재함

4-(1). Checkpointer

LangGraph은 thread state 영속성을 checkpointer 인터페이스로 추상화



Checkpointer 책임 :

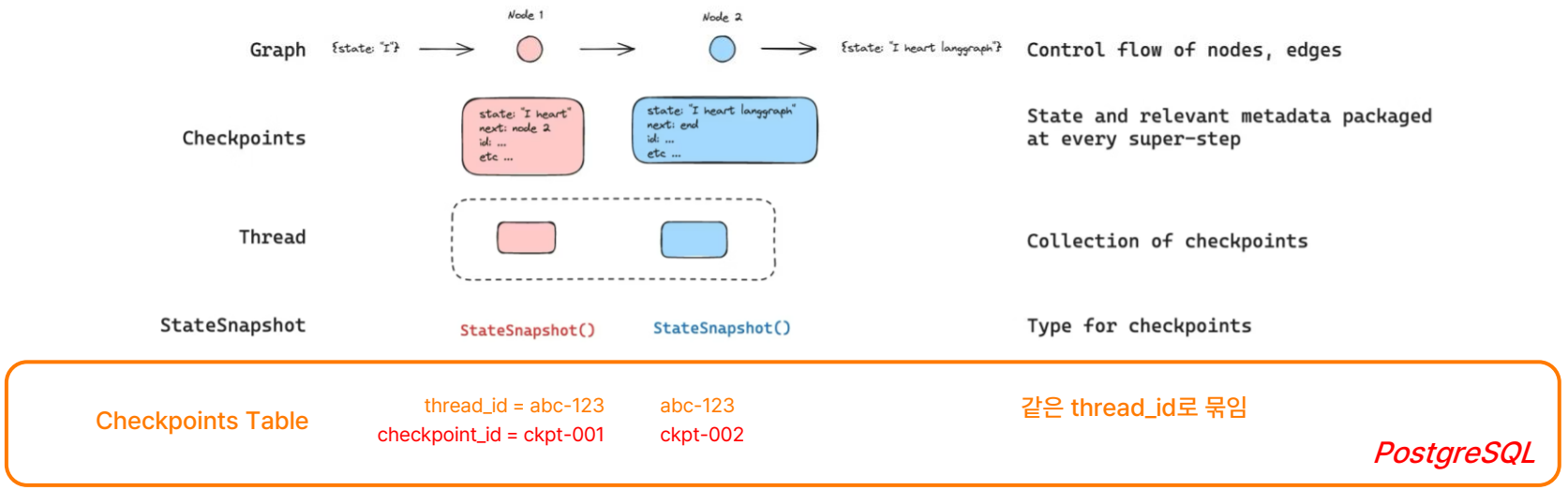
- Graph의 매 Step 마다 state snapshot(checkpointer) 저장
- thread_id로 식별
- 같은 thread_id로 다시 요청하면 이전 checkpointer로 복원
- Time travel 지원 : 특정 checkpointer로 되돌아가기

※ 구현체 :
MemorySaver, SqliteSaver, PostgresSaver,
AsyncPostgresSaver, RedisSaver, etc.

[참고] AsyncPostgresSaver

- Async : FastAPI가 async backend로 동일한 async 패러다임 유지
- PostgreSQL : production-grade + multi-instance 공유 가능

- AsyncPostgresSaver :
  - 매 Graph step의 state snapshot을 PostgreSQL 테이블에 저장
  - 같은 thread_id로 재요청 시 PostgreSQL에서 복원

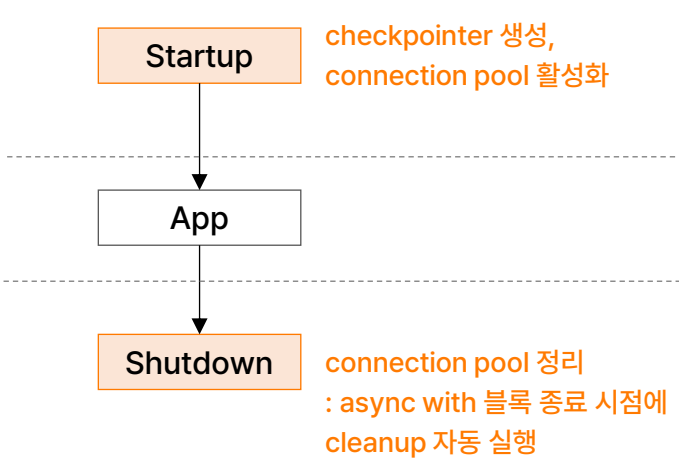


Source : https://docs.langchain.com/oss/python/langgraph/persistence (modified)

4-(2). Lifespan

- 프로그램이 켜질 때 한번 준비하고, 꺼질 때 한번 정리하는 구간 (비유 : 개점 준비 ~ 폐점 정리)
- Checkpointer는 connection pool을 가지는 resource (DB 연결)
- App 시작 시 한번 생성, 종료 시 정리되어야 함 → Request 마다 생성하면 비효율 + 누수
- Singleton 패턴 – FastAPI 권장 방식

- Lifespan 동작 :




```python
@asynccontextmanager
async def lifespan(app):
    async with AsyncPostgresSaver.from_conn_string(...) as cp:
        app.state.checkpointer = cp

        yield

    # async with 종료
```

4-(3). Checkpointer 외부 주입

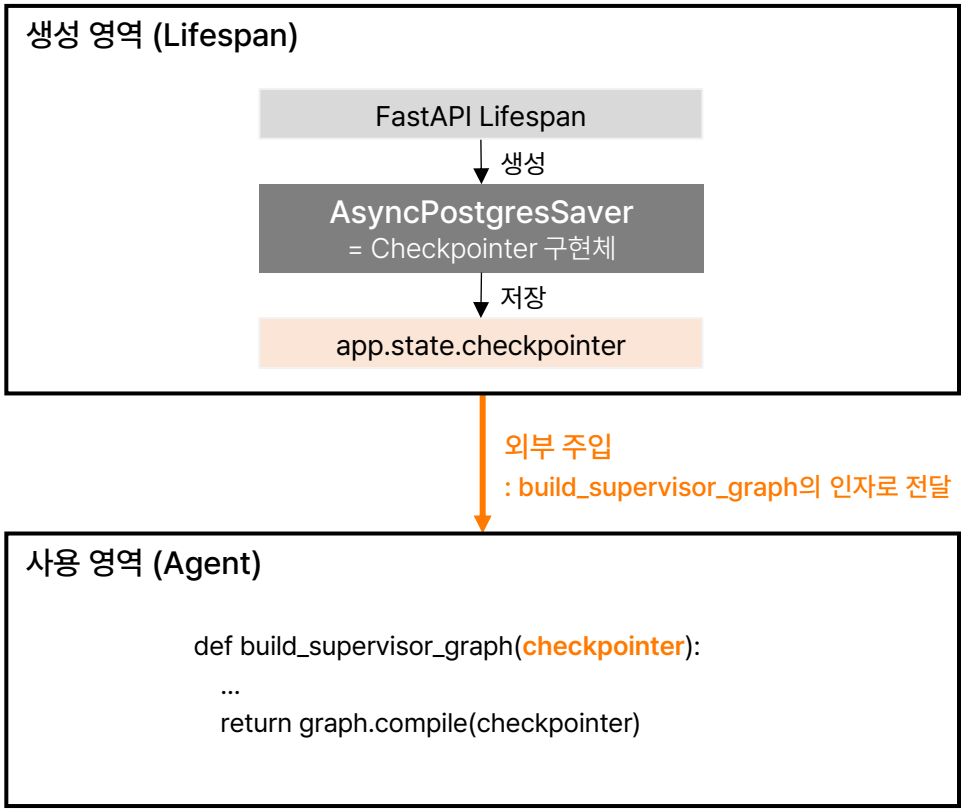
- Lifespan이 생성한 checkpointer를 에이전트에게 전달해야 함
- 에이전트는 checkpointer를 인자로 받는 구조로 변경하면 처리 가능

```
# 현재 코드

def build_supervisor_graph():
    ...
    checkpointer = MemorySaver()
    return graph.compile(checkpointer)
```

```
# 업데이트 코드

def build_supervisor_graph(checkpointer):
    ...
    # checkpointer 외부에서 받음
    return graph.compile(checkpointer)
```

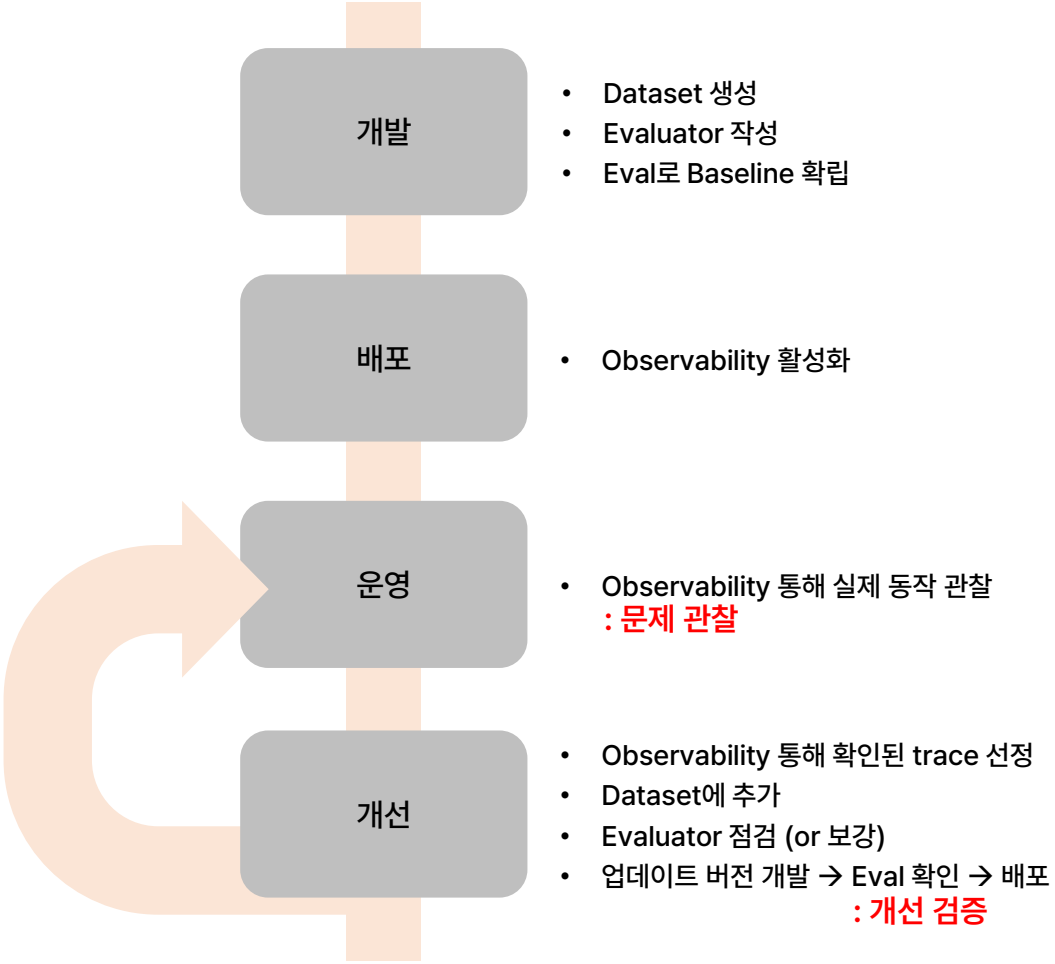


4. 통합 환경의 운영

SUMMARY

SUMMARY

1. Observability & Eval



- Dataset 생성
- Evaluator 작성
- Eval로 Baseline 확립

- Observability 활성화

- Observability 통해 실제 동작 관찰
  : 문제 관찰

- Observability 통해 확인된 trace 선정
- Dataset에 추가
- Evaluator 점검 (or 보강)
- 업데이트 버전 개발 → Eval 확인 → 배포
  : 개선 검증

❑ **Observability와 Eval은 분리될 수 없는 한 쌍 입니다.**

- Observability만 있으면 변경이 좋은지 확신 할 수 없음
- Eval만 있으면 운영의 실제 문제를 대응 할 수 없음

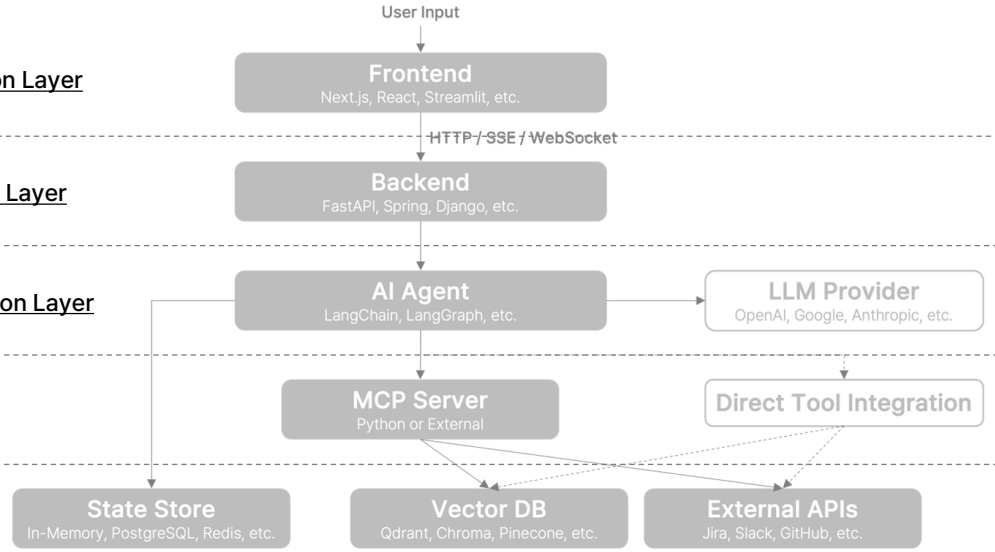
➔ 두 가지가 함께 동작할 때, AI Agent Product은 점진적으로 개선됨

SUMMARY

2. Error Handling

- 하위 Layer → 상위 Layer : 무엇이 잘못 되었는지 명확하게 전달
- 상위 Layer : 전달받은 Error를 현재 Context에서 판단
- 최상위 Layer (Frontend) : 사용자에게 전달되는 메시지 표현 책임

Layer :



Error Responsibility :

- 사용자에게 명확하게 전달
- Retry UX, Disconnection 처리 등

- HTTP 응답 형식화 및 사용자 친화적 형태로 변환
- Stream 중간 에러 처리

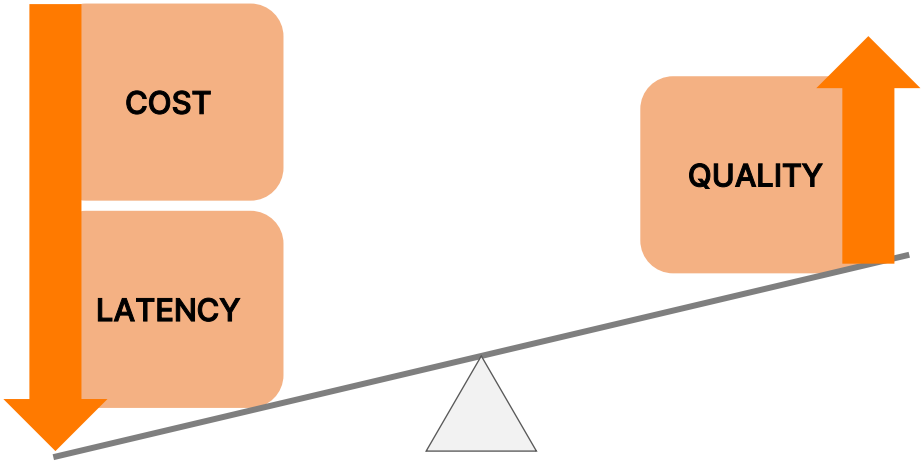
- Sub-agent 또는 Tool 실패 시 fallback 결정
- Partial result로 사용자에게 전달 가치 판단

- MCP Server와의 통신 관리 : Connection 복구, Timeout 처리
- 실패 시 Tool 단위 실패 내용을 상위 전달

- 외부 시스템 통신 Error 발생 → Retry 1차 시도
- 자체 해결 불가 시 Error Type 상위 전달

SUMMARY

3. Cost & Latency



Agent - 통합시스템에서의 특수성 :

- 호출이 누적됨 :
  - Single agent : LLM 호출 1~3회 수준
  - Multi-agent : LLM 호출 (최소) 5+ 수준
  - Caching : Cost ↓, Latency ↓ → Quality ↑ (일관성 측면)
- Tool 호출이 LLM 호출을 추가적으로 요구함 :
  - Tool call → Tool 응답 → 그 응답을 처리하는 LLM 호출
  - 하나의 tool 호출이 LLM 호출 2번 발생
  - Tool 호출 감소 : Cost ↓, Latency ↓ → Quality ↓ (정보 부족)
- Context 누적 :
  - LLM 호출이 이전 메세지를 모두 포함
  - Multi-turn이 길어질수록 input token 증가
  - Context 압축 : Cost ↓, Latency ↓ → Quality ↓
- Latency 누적 :
  - 순차 실행 시 latency는 각 단계 latency의 총합
  - 한 단계만 느려져도 전체 서비스 지연과 연결
- Sub-agent 효율성 :
  - 전체 응답만 보면 sub-agent 비효율성을 파악하기 어려움 → Observability
  - 병렬 실행 : Latency ↓ → Cost 변화 거의 없음

SUMMARY

4. Stateless Session

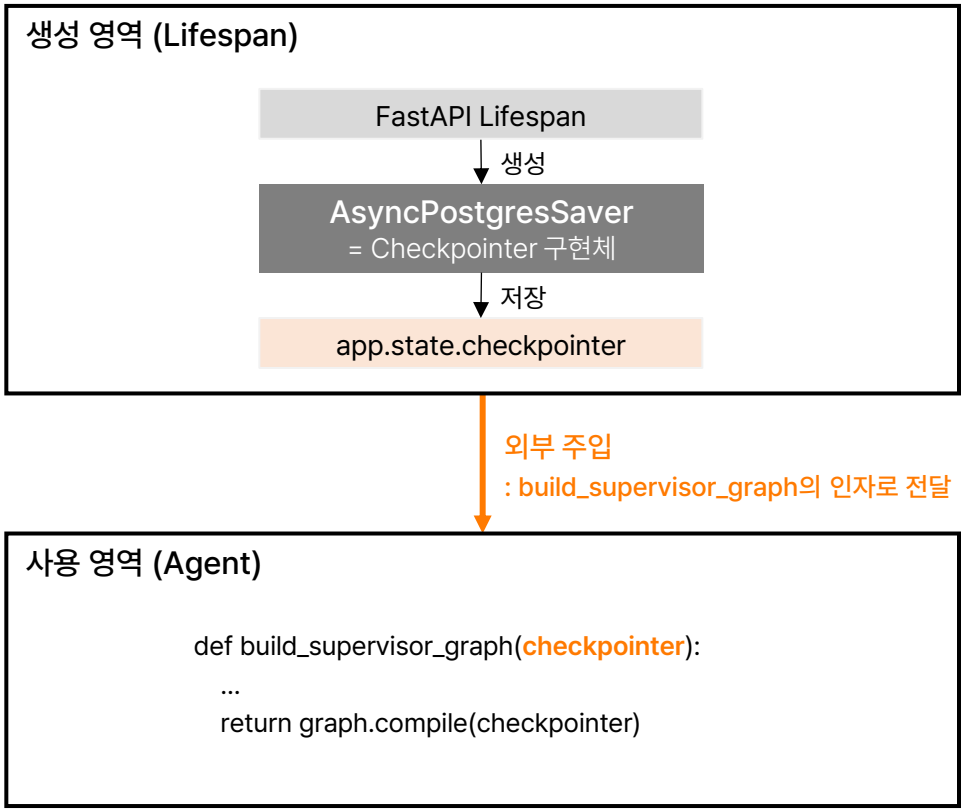
- Lifespan이 생성한 checkpointer를 에이전트에게 전달해야 함
- 에이전트는 checkpointer를 인자로 받는 구조로 변경하면 처리 가능

```
# 현재 코드

def build_supervisor_graph():
    ...
    checkpointer = MemorySaver()
    return graph.compile(checkpointer)
```

```
# 업데이트 코드

def build_supervisor_graph(checkpointer):
    ...
    # checkpointer 외부에서 받음
    return graph.compile(checkpointer)
```



Hands-On

Notion > Observability & Eval

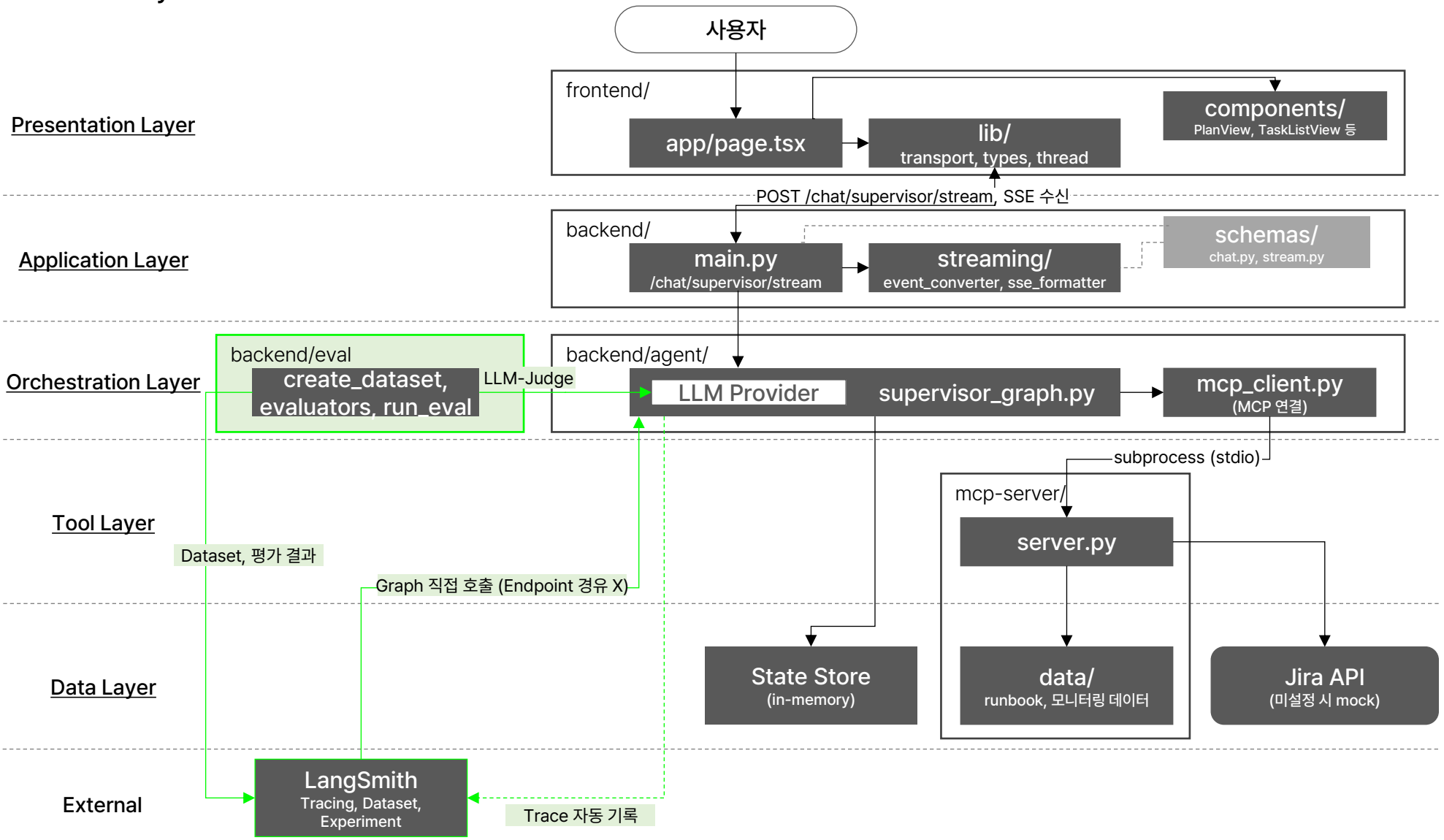
- ❑ Notion 가이드를 확인하고,
- ❑ 코드와 LangSmith 통해 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 해당 내용을 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인합니다.

※ 이제부터 다룰 문제들은 Observability 도구로 진단 및 측정하며 진행합니다 !!

Hands-On

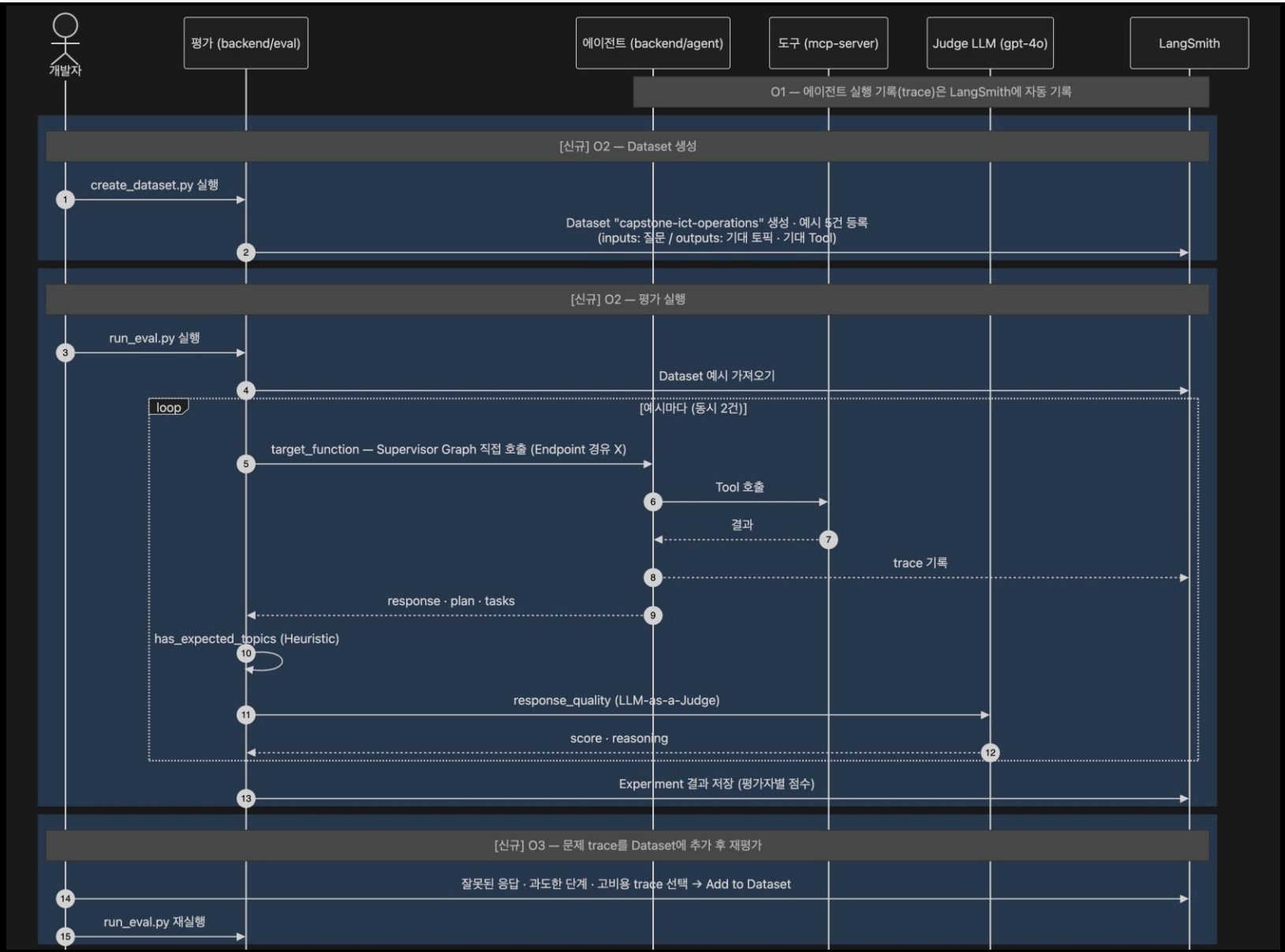
모듈 연결도



Hands-On

서비스 흐름도

Observability & Eval



Hands-On

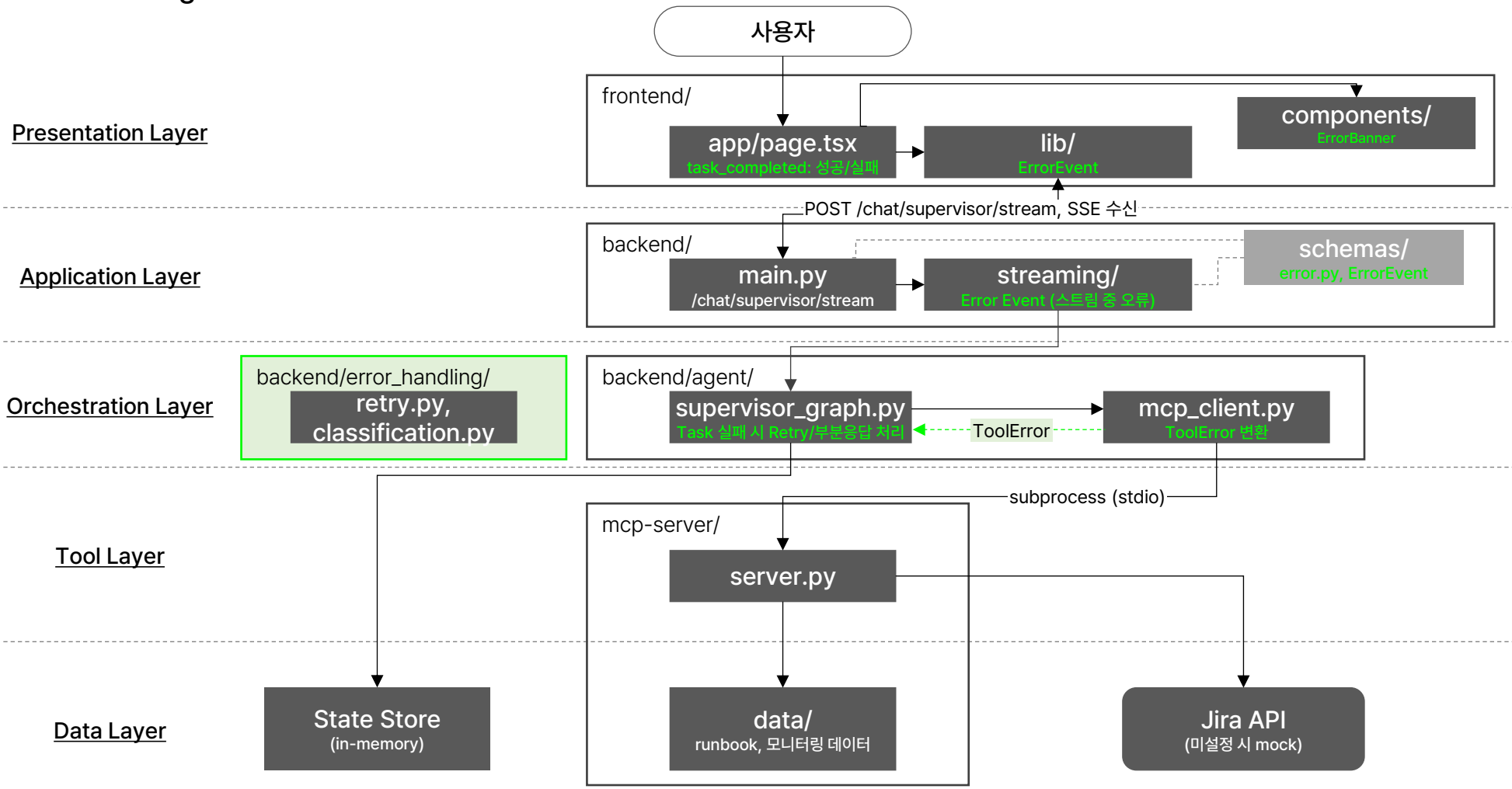
Notion > Error Handling

- ❑ Notion 가이드를 확인하고,
- ❑ 터미널과 LangSmith 통해 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 해당 내용을 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인 합니다.

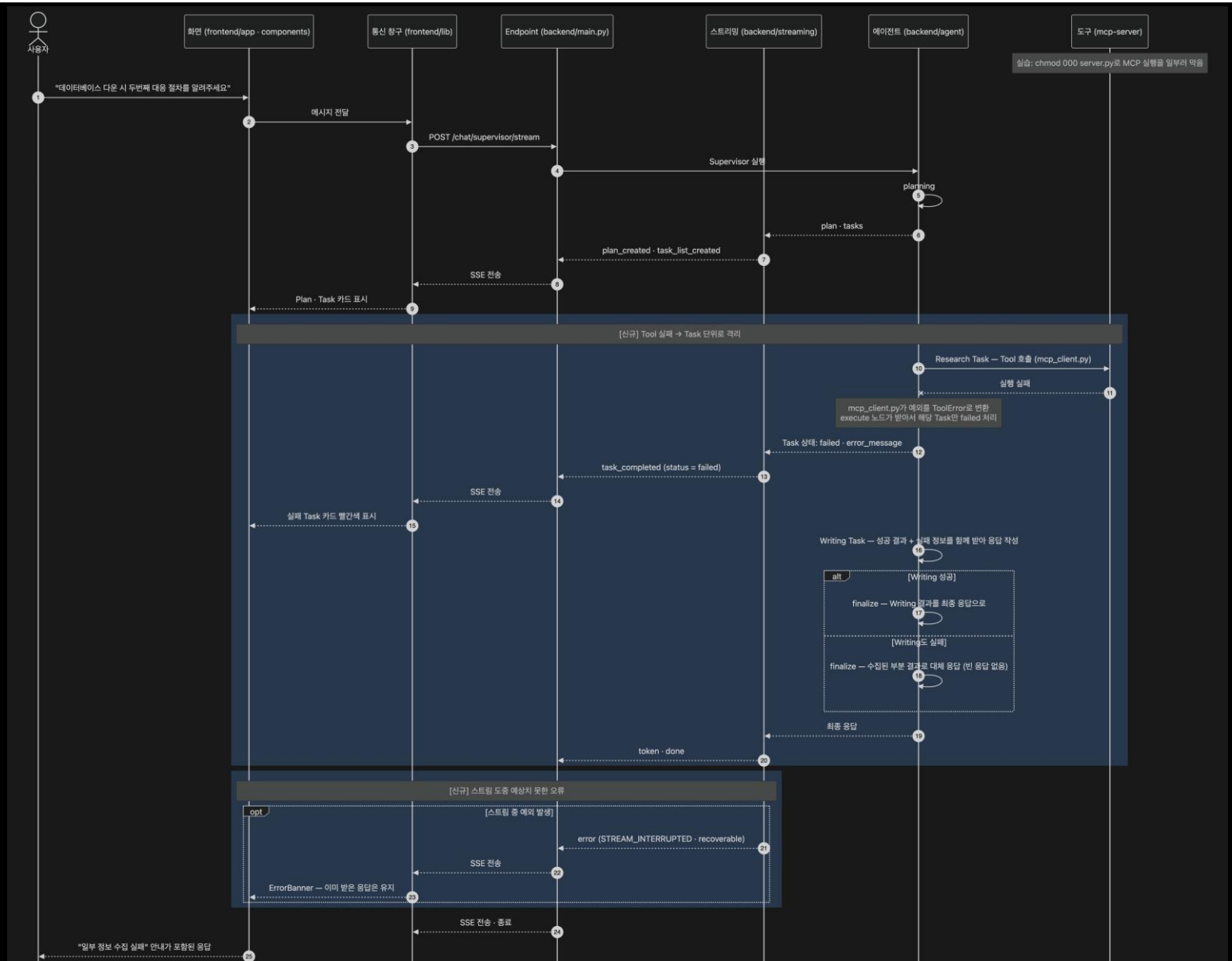
모듈 연결도

Error Handling



서비스 흐름도

Error Handling



Hands-On

Notion > Cost & Latency

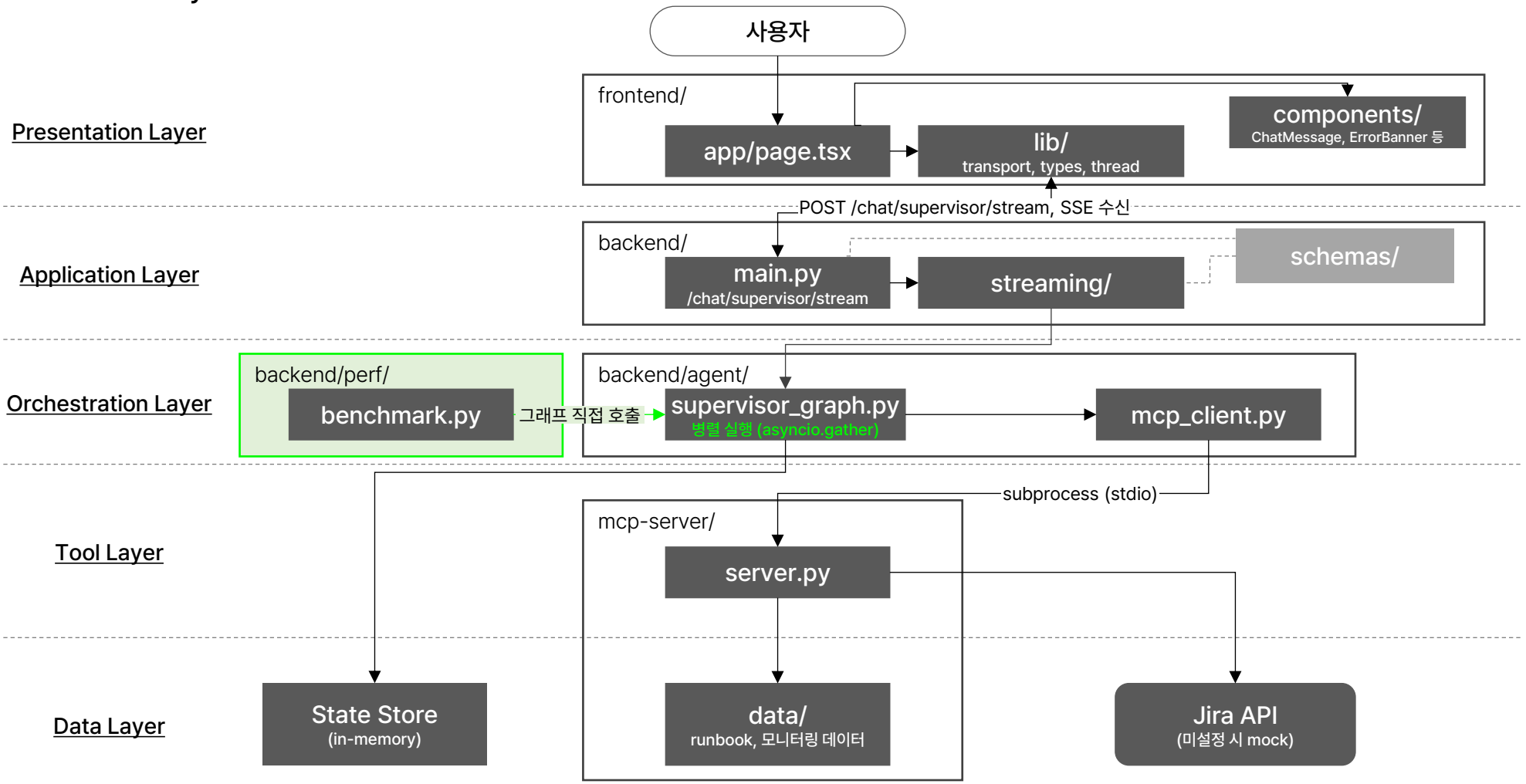
- ❑ Notion 가이드를 확인하고,
- ❑ 터미널과 LangSmith 통해 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 해당 내용을 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인합니다.

모듈 연결도

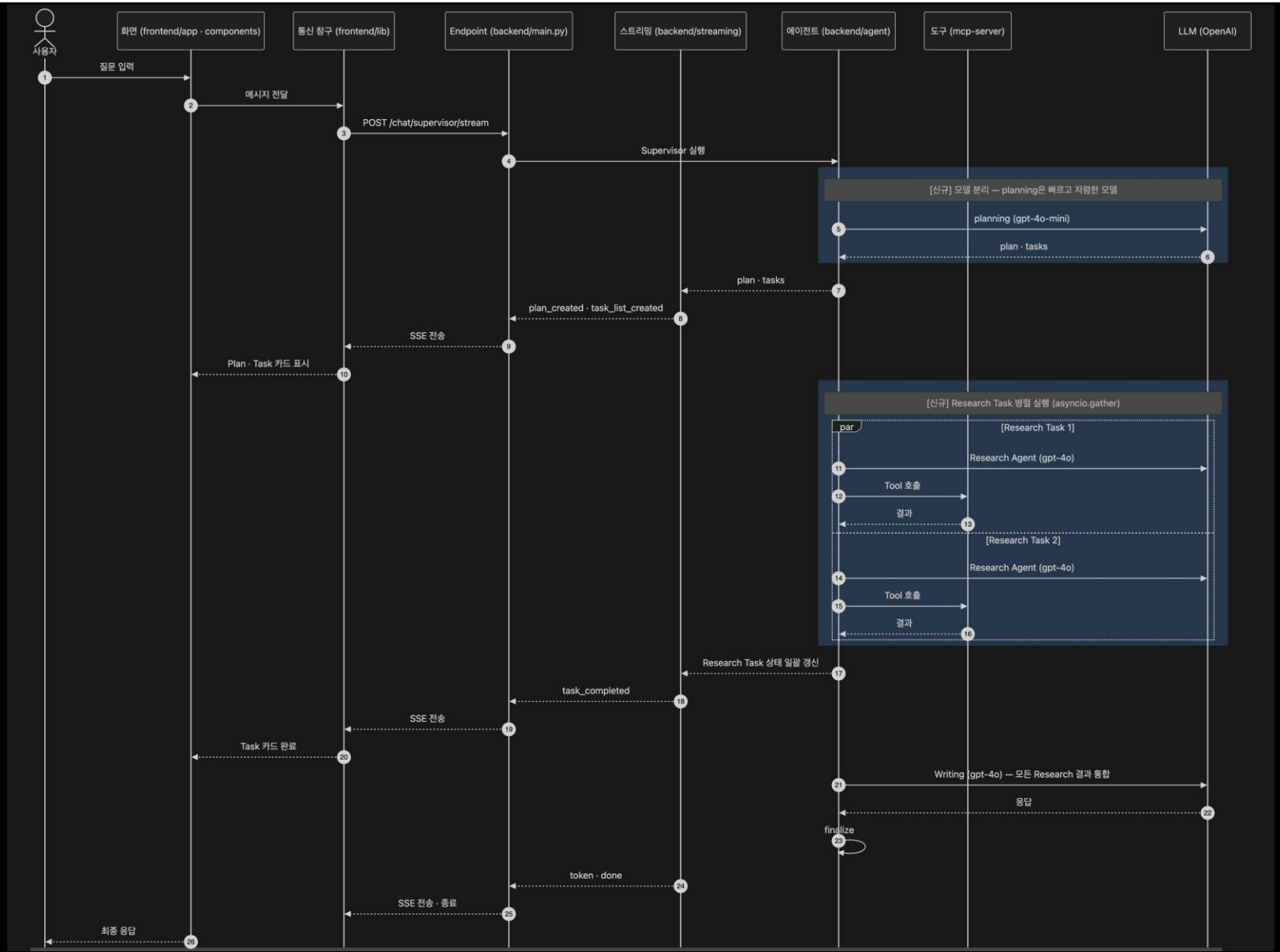
Hands-On

Cost & Latency



서비스 흐름도

Cost & Latency



Hands-On

Notion > Stateless Session

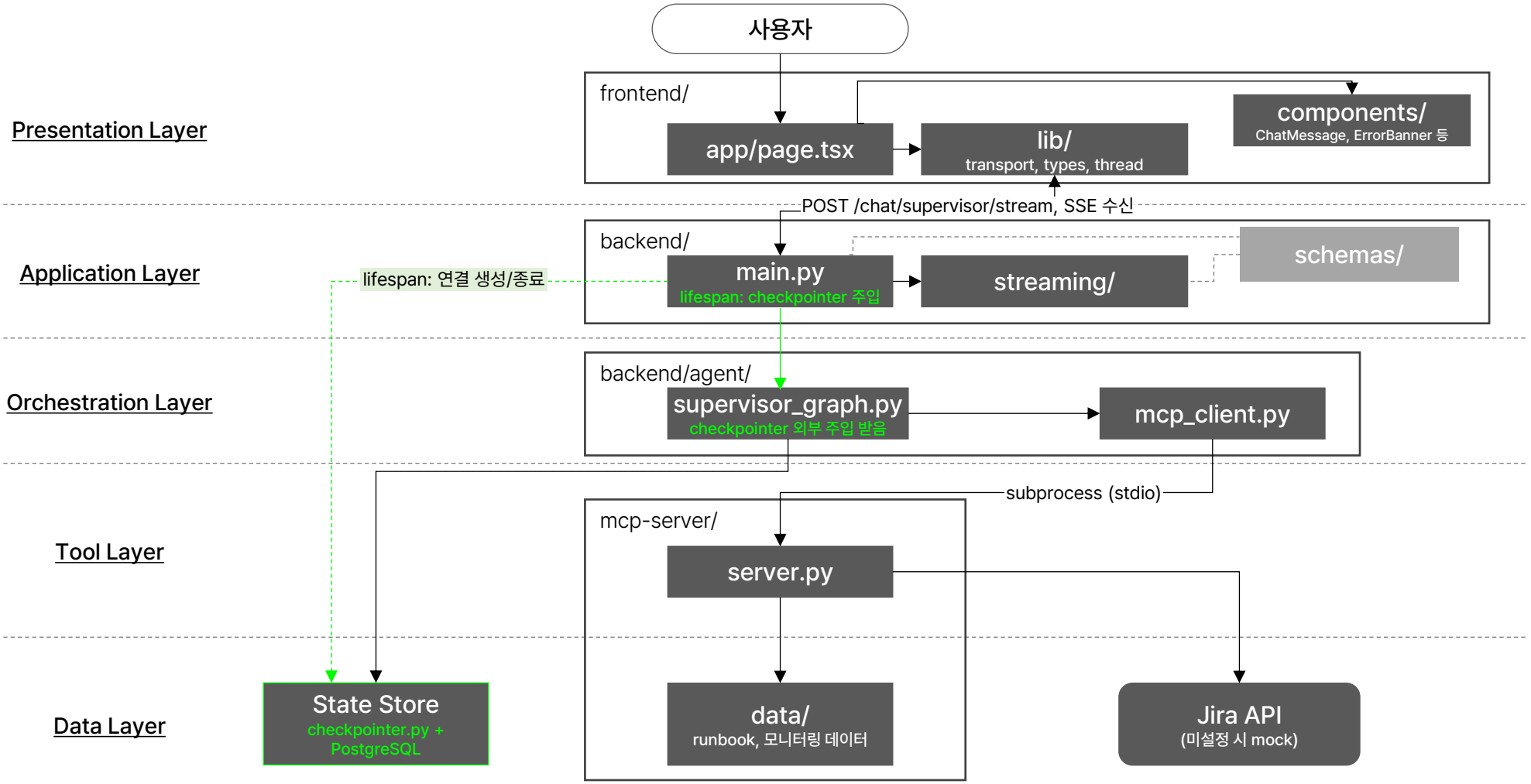
- ❑ Notion 가이드를 확인하고,
- ❑ 터미널과 LangSmith 통해 결과를 확인 합니다.

- Notion의 Guide 항목을 확인하고 해당 내용을 코드 파일에 반영합니다.
- 동작 확인 및 검증 챕터의 내용을 확인합니다.

Hands-On

모듈 연결도

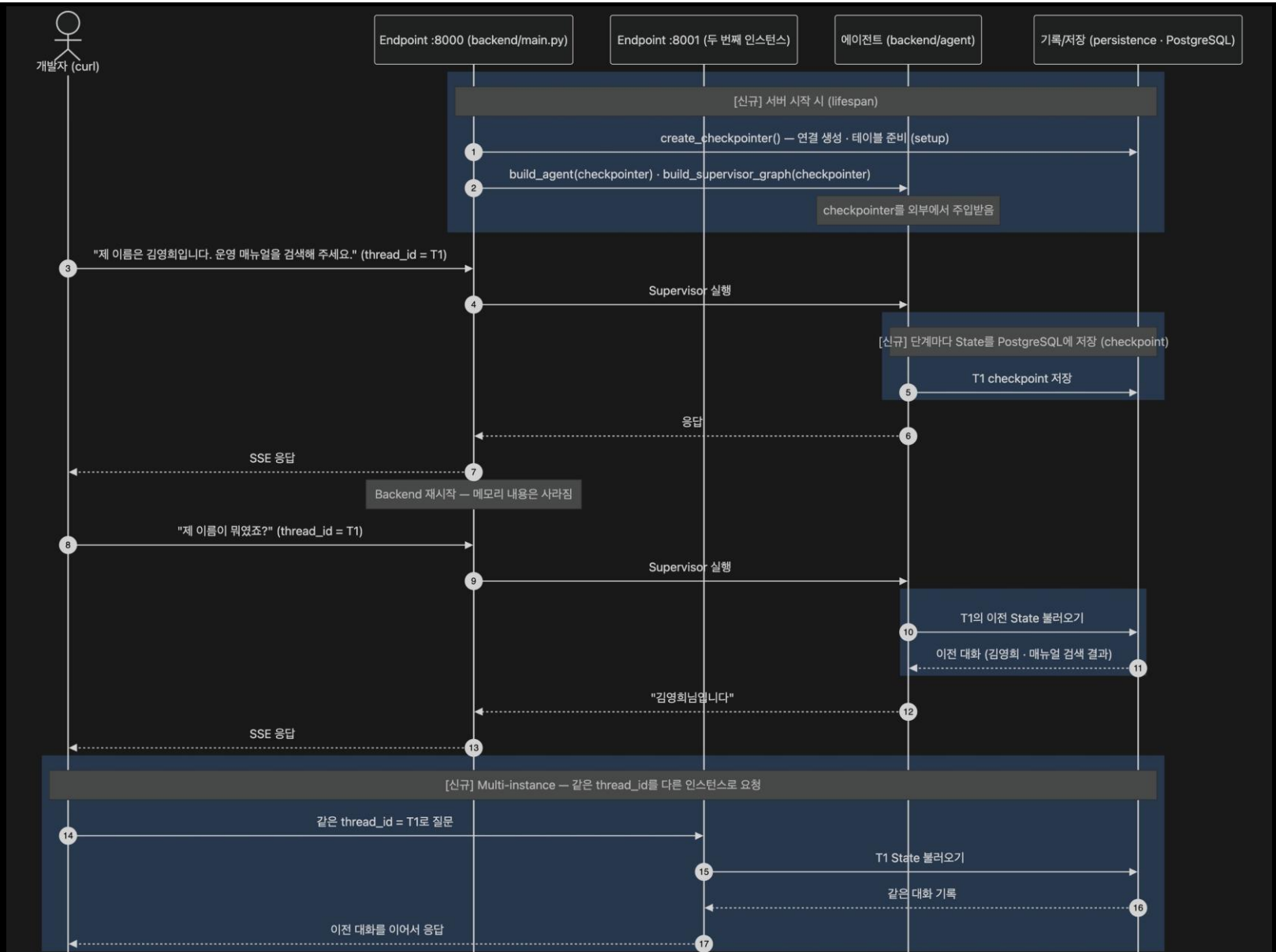
Stateless Session



Hands-On

서비스 흐름도

Stateless Session





수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX